

Sunburn device: the whole program

every block of the program that runs on the wrist, explained in plain English · printed 2026-09-15

The Sunburn device is a wearable built on a BBC micro:bit, a small computer with 25 lights and two buttons. It measures the sun's UV with a sensor, checks the online UV index for where you are through a phone, and then flashes, beeps or taps your arm until you put sunscreen on. Two hours later it reminds you to reapply. The program that does all this is built from 76 coloured blocks. Every one of them is on these sheets, with a note saying what it does.

Each picture is one block of the program, exactly as it looks in the MakeCode editor where the program is built by snapping coloured pieces together. A block is read from top to bottom. Under each picture: the block's name, what kind of block it is, what it does, which other blocks use it (Used by) and which blocks it calls on (Uses). This sheet is the whole program at a glance: nine numbered sections. Sections 1 and 2 are the plan the device follows; sections 3 to 9 are the parts that plan calls on.



- 1

Start here: settings · 1 block · sheet 2
This block runs first, the moment the device is switched on. It holds every setting: which UV sensor is fitted, how the sound and the tapping arm are wired, the UV levels that count as low, moderate, very high and extreme, and how long each wait is. Then it starts everything up. To change how the device behaves, this is the only place to look.
- 2

The flowchart · 20 blocks · sheet 3
The plan the device follows all day, taken straight from the Sunburn Flowchart. Every box and every question on the flowchart is one block here, with the same name, in the same order: read the UV sensor, check the reading makes sense, ask the phone for the online UV index, decide how strong the sun is, then react. The forever block keeps the plan running round and round.
- 3

Buttons · 3 blocks · sheet 7
The micro:bit has two buttons. A means done: sunscreen is on, or I am going inside. B wakes the device up. Pressing both together turns it off, or on again.
- 4

Alerts · 4 blocks · sheet 8
The ways the device gets your attention: flashing and beeping, tapping your arm with the small motor, and going to sleep when nobody answers. Each one keeps going until you press A or its time limit runs out. No answer for a long time means the device is probably not being worn, so it sleeps until a button is pressed.
- 5

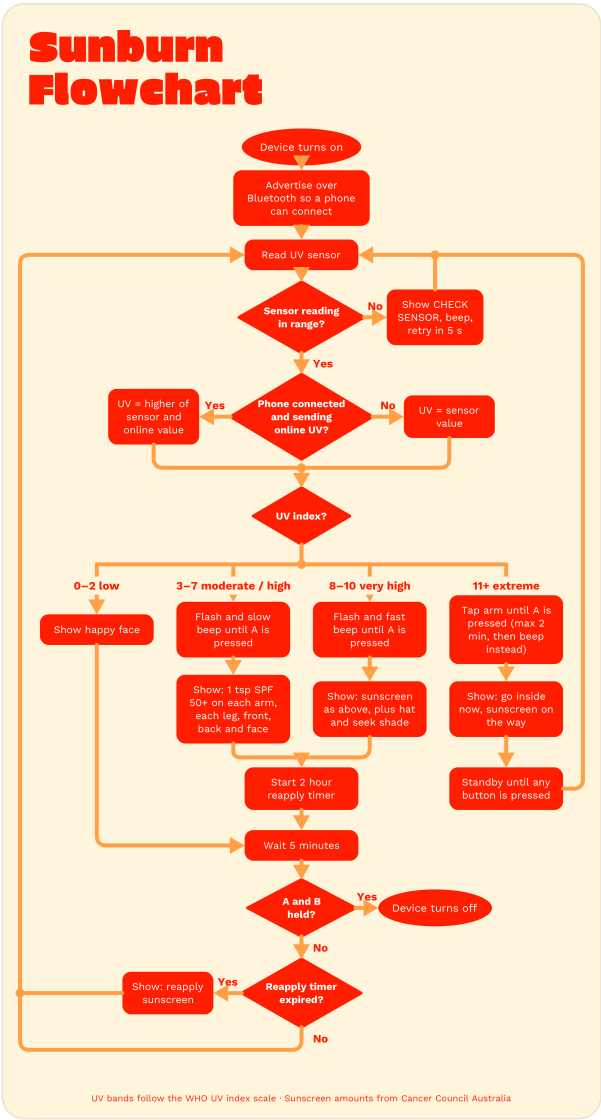
Screen and lights · 8 blocks · sheet 9
What you see. The 5 by 5 grid of lights shows the UV number, a happy face or a warning, and messages one word at a time. If a small screen is fitted, it shows the number large, the level, the message and the countdowns.
- 6

Sound and the tapping arm · 5 blocks · sheet 11
The beeps, and the small motor (a servo) whose arm taps your wrist when the sun is extreme.
- 7

Talking to the phone · 12 blocks · sheet 11
How the wearable and the phone app talk over Bluetooth. The phone sends the online UV index for where you are; the wearable sends back what it is doing every two seconds and reports each event, such as sunscreen applied. The same messages also go down the USB cable, so everything can be tested on a laptop. Every message ends with a checksum, so a message damaged on the way is thrown away instead of being misread.
- 8

Helpers · 11 blocks · sheet 14
Small jobs the other blocks share: the maths for each type of sensor, turning minutes into the milliseconds the program counts in, sorting numbers, tidying up numbers, times and text, and the checksum that goes on the end of every message.
- 9

The optional screen · 12 blocks · sheet 16
Only used if a small screen is added to the wearable. These blocks draw letters and numbers on it dot by dot. The device works exactly the same without a screen.



The Sunburn Flowchart the device follows. Every box on it is a block in section 2 with the same name.

What the block colours mean

- | | |
|-------------------------------|-----------------------------|
| Variables: remember a value | Text |
| Functions: a step with a name | Arrays: lists |
| Basic: the lights, pauses | Music: beeps |
| Input: buttons and time | Pins: sensor, motor, screen |
| Loops: repeat | Bluetooth: the phone |
| Logic: if, compare | Serial: the USB cable |
| Maths | Control: timing |

Reading the other sheets

Every picture is one block from the editor at 60% of its size on screen. Under it: the block's name, what kind of block it is, what it does, which blocks use it (Used by) and which blocks it calls on (Uses).

UV index How strong the sun's burning rays are, from 0 (none) to 11 and above (extreme). Protection is needed from 3 upwards, even on cool or cloudy days.

micro:bit The small computer the program runs on. It has a 5 by 5 grid of lights, buttons A and B, and sockets called pins (P0, P1, P2) that other parts plug into.

Block One instruction of the program. Blocks snap together and run in order from top to bottom. A block with a name at the top (function) is a step that other blocks can call on.

Sensor The part that measures UV. It gives a voltage, which the program turns into a UV index number.

Servo A small motor with an arm. Here it taps the wearer's wrist when the sun is extreme.

Bluetooth and USB The two ways the wearable talks to a phone or a laptop. The phone app sends the online UV index; the wearable sends back what it is doing.

OLED An optional small screen. Without one, the grid of lights does the job.

ms and Hz ms are milliseconds, thousandths of a second. Hz sets how high a beep sounds.

Demo timings Much shorter waits (seconds instead of minutes and hours) so the whole day can be shown in a few minutes at a demo.

Names in the blocks Words like uvSampleCount are the program's own names for its settings and memory. Read them as English: uv sample count is how many readings to take.

1

Start here: settings

1 block

This block runs first, the moment the device is switched on. It holds every setting: which UV sensor is fitted, how the sound and the tapping arm are wired, the UV levels that count as low, moderate, very high and extreme, and how long each wait is. Then it starts everything up. To change how the device behaves, this is the only place to look.

on start

set uvSensorType to 1

set uvSampleCount to 15

set soundOutput to 1

set soundVolume to 180

set servoType to 1

set servoRestAngle to 90

set servoTapAngle to 30

set oledMode to 1

set oledType to 1

set oledAddress to 60

set uvLowMax to 2

set uvHighMax to 7

set uvVeryHighMax to 10

set waitMinutes to 5

set reapplyHours to 2

set tapMaxMinutes to 2

set alertGiveUpMinutes to 10

set onlineUvMaxAgeMinutes to 30

set sensorRetrySeconds to 5

set usbSpeed to 1

set firmwareVersion to "3.2"

call applySensorPreset

call applyTimings

call setupSound

call setupOled

call setupUsbLink

call advertiseOverBluetooth

if button B is pressed then

set demoMode to true

call applyTimings

call deviceTurnsOn

on start Runs once, when the device is switched on

The first thing that runs when the device is switched on. All the settings are listed here, then the start-up steps: work out the sensor maths, set the timings, get the sound, the screen and Bluetooth ready, and show that the device is on. Holding B while switching on uses the fast demo timings.

Uses: deviceTurnsOn, setupSound, setupUsbLink, advertiseOverBluetooth, applySensorPreset, applyTimings, setupOled

2 The flowchart

20 blocks

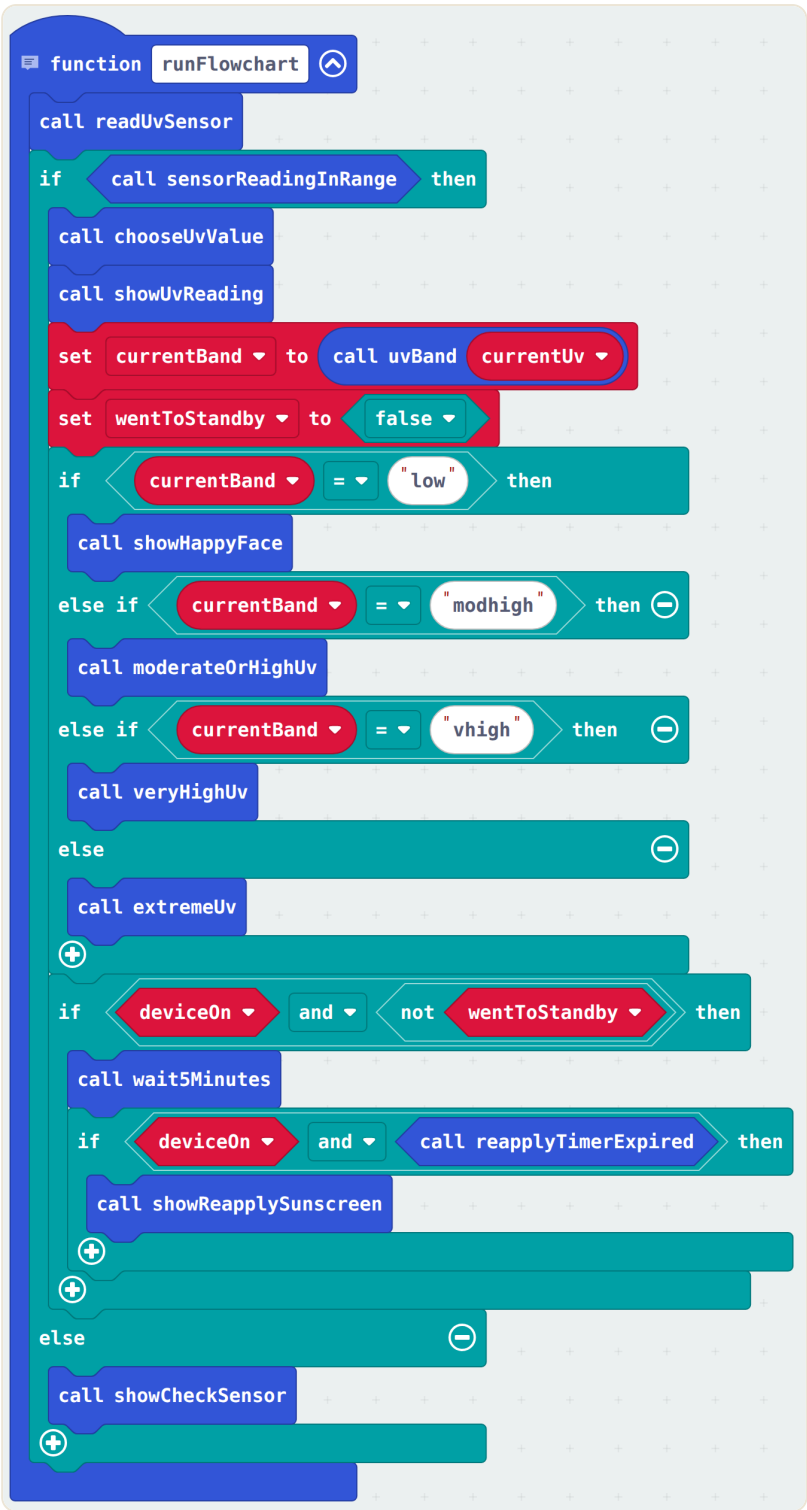
The plan the device follows all day, taken straight from the Sunburn Flowchart. Every box and every question on the flowchart is one block here, with the same name, in the same order: read the UV sensor, check the reading makes sense, ask the phone for the online UV index, decide how strong the sun is, then react. The forever block keeps the plan running round and round.



forever Runs again and again, all day

Runs the flowchart once, then goes straight back to the top and runs it again, all day long.

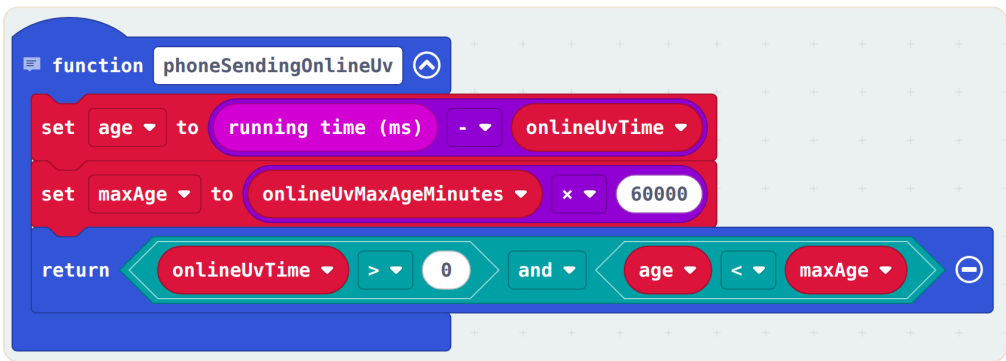
Uses: runFlowchart



runFlowchart A step of the program

The whole flowchart from top to bottom. Read the sensor. If the reading makes sense, pick the UV value, show it, decide the level and react: a happy face, or an alert until A is pressed. Then wait 5 minutes and check whether it is time for more sunscreen. If the reading does not make sense, show CHECK SENSOR instead. Pressing A and B together at any moment turns the device off and stops every wait.

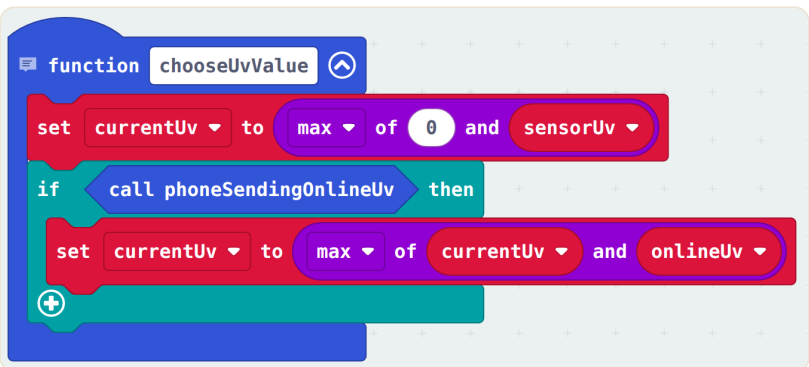
Used by: forever · **Uses:** readUvSensor, sensorReadingInRange, showCheckSensor, chooseUvValue, uvBand, showHappyFace, moderateOrHighUv, veryHighUv, extremeUv, wait5Minutes, reapplyTimerExpired, showReapplySunscreen, showUvReading



phoneSendingOnlineUv A yes-or-no question the program asks

Asks: has the phone app sent an online UV index in the last 30 minutes?

Used by: chooseUvValue, showUvReading, sendStatus



chooseUvValue A step of the program

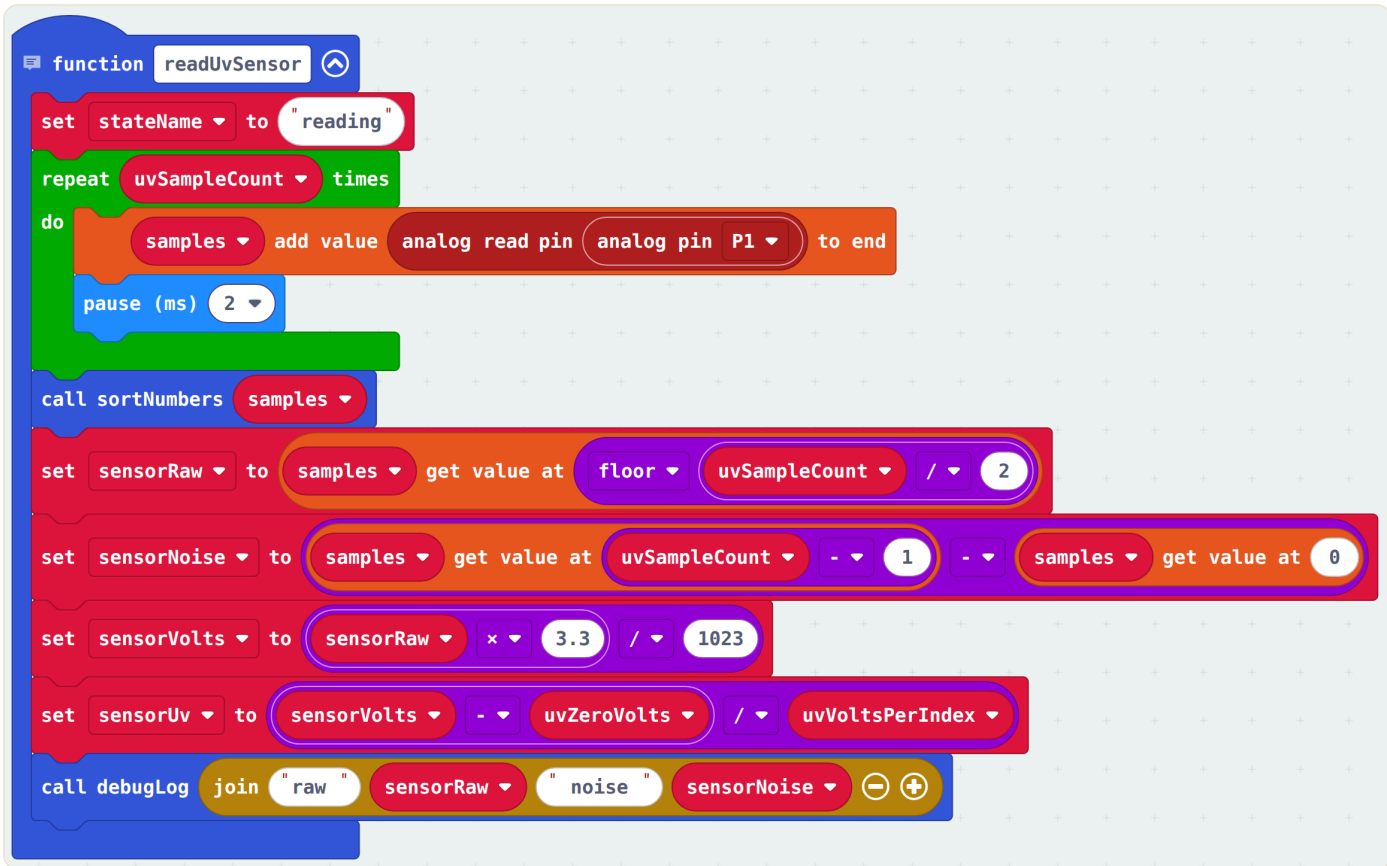
Picks the UV number the device will act on: the higher of the sensor's reading and the phone's online value, or just the sensor's reading when the phone is not sending.

Used by: runFlowchart · **Uses:** phoneSendingOnlineUv

2 The flowchart (continued)

20 blocks

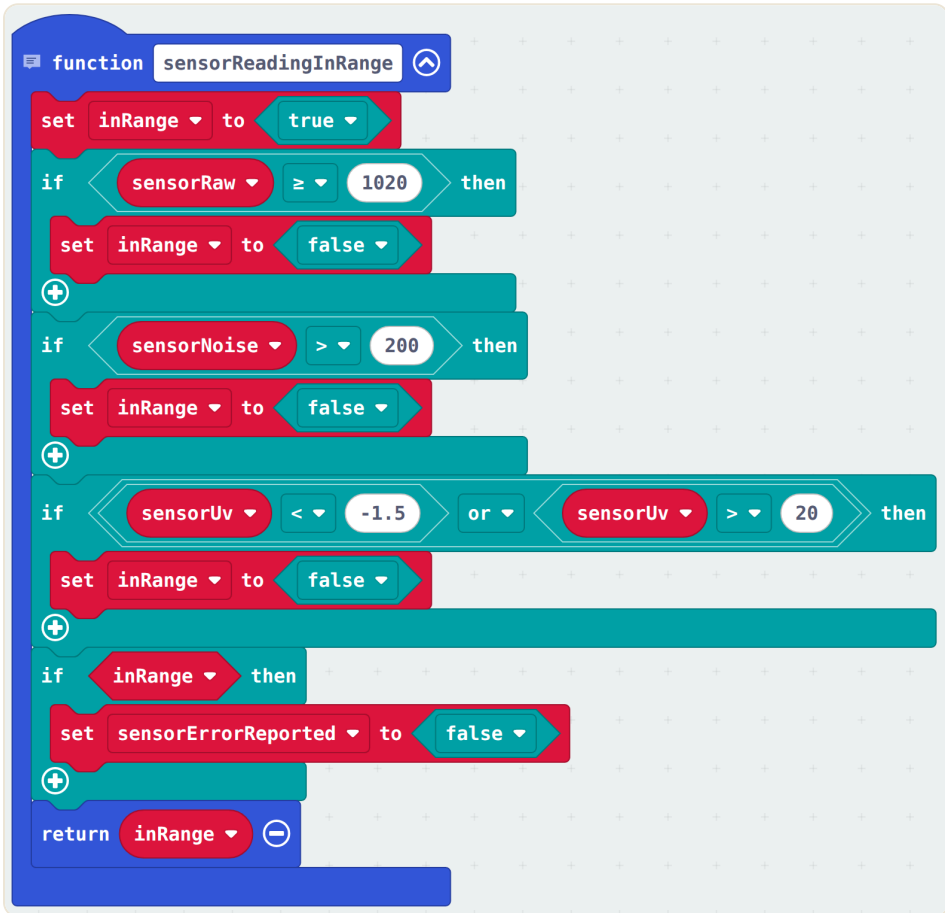
The plan the device follows all day, taken straight from the Sunburn Flowchart. Every box and every question on the flowchart is one block here, with the same name, in the same order: read the UV sensor, check the reading makes sense, ask the phone for the online UV index, decide how strong the sun is, then react. The forever block keeps the plan running round and round.



readUvSensor A step of the program

Reads the UV sensor plugged into pin P1. It takes 15 quick readings and uses the middle one, so a single odd reading cannot set off a false alarm. Then it turns the sensor's voltage into a UV index number.

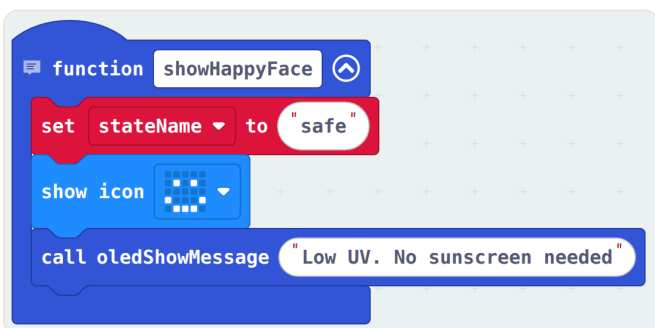
Used by: runFlowchart, handleCommand, calibrateToKnownUv · **Uses:** sortNumbers, debugLog



sensorReadingInRange A yes-or-no question the program asks

Asks: is this reading believable? The answer is no if the sensor is missing or shorted, if the readings jump about (a loose wire), or if the number could not be real.

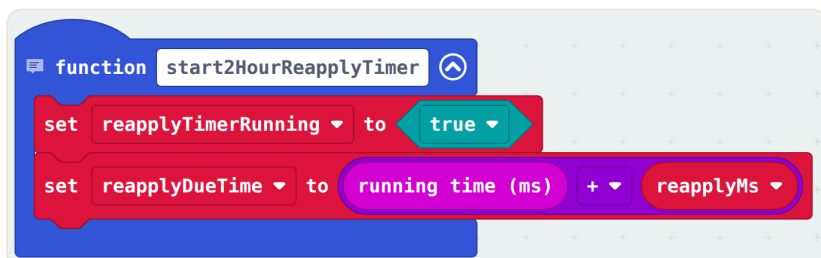
Used by: runFlowchart



showHappyFace A step of the program

UV 0 to 2: shows a happy face. No sunscreen needed right now.

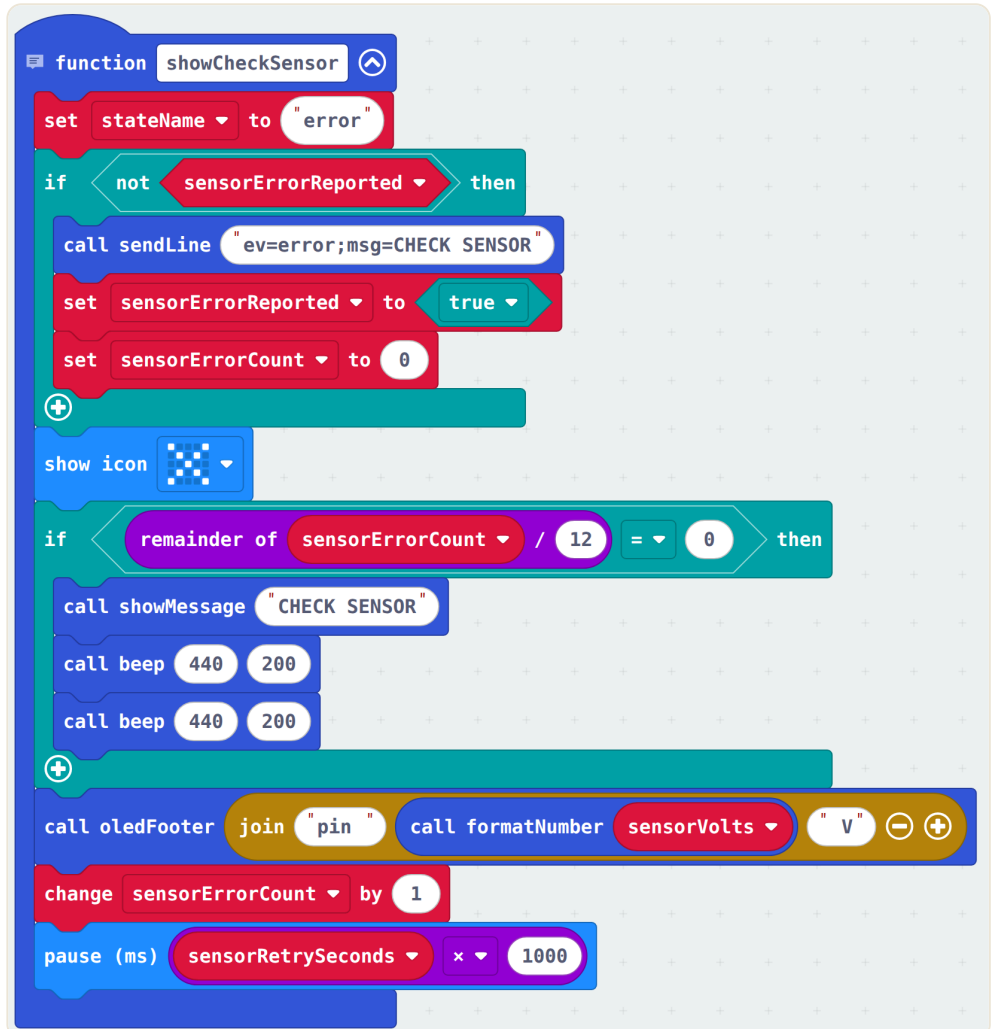
Used by: runFlowchart · **Uses:** oledShowMessage



start2HourReapplyTimer A step of the program

Starts the 2 hour countdown to the next sunscreen.

Used by: sunscreenApplied



showCheckSensor A step of the program

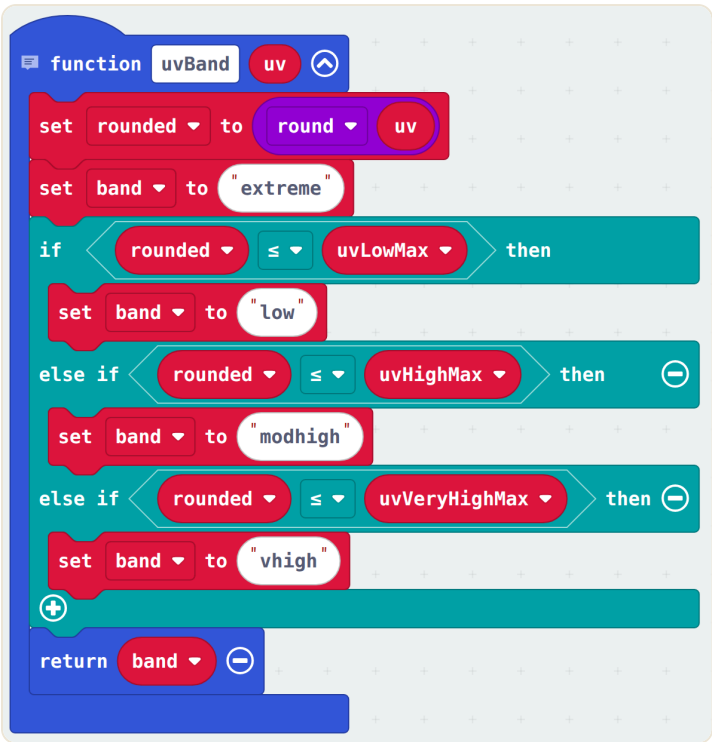
When the reading is not believable: shows a cross, and the first time scrolls CHECK SENSOR and beeps twice, then again about once a minute while the fault lasts. It tells the app once, and tries again every 5 seconds. A faulty sensor is never treated as safe.

Used by: runFlowchart · **Uses:** showMessage, oledFooter, beep, sendLine, formatNumber

2 The flowchart (continued)

20 blocks

The plan the device follows all day, taken straight from the Sunburn Flowchart. Every box and every question on the flowchart is one block here, with the same name, in the same order: read the UV sensor, check the reading makes sense, ask the phone for the online UV index, decide how strong the sun is, then react. The forever block keeps the plan running round and round.



uvBand Works out a piece of text · needs: uv

Turns a UV index into one of the four levels on the flowchart: low (0 to 2), moderate or high (3 to 7), very high (8 to 10) or extreme (11 and above).

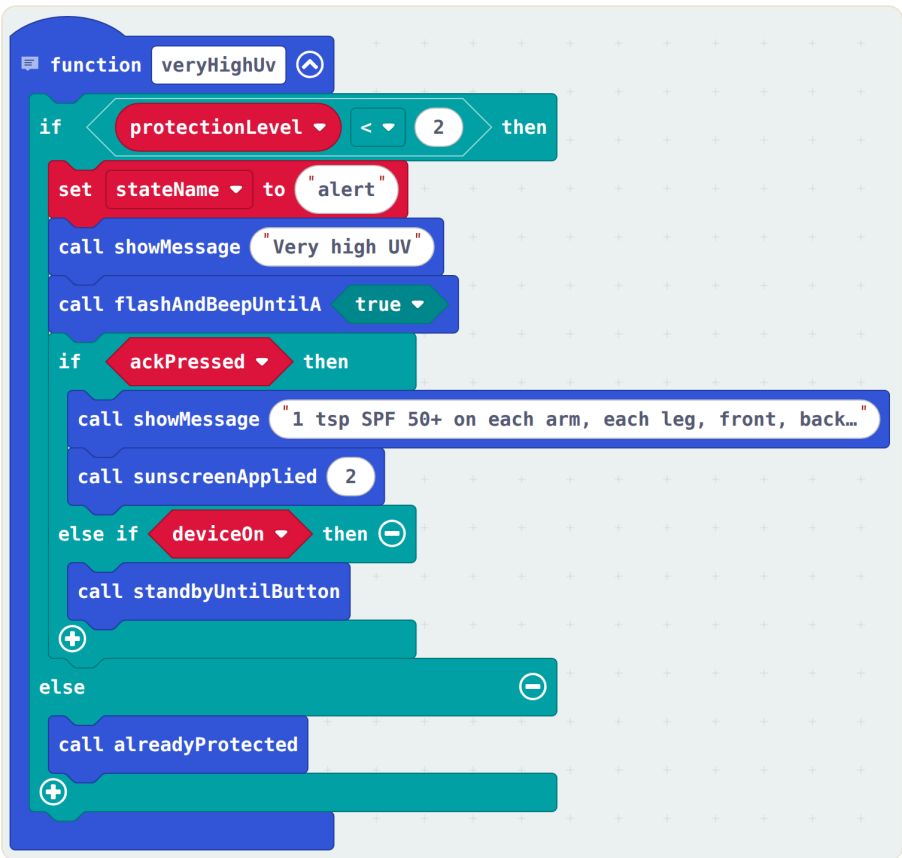
Used by: runFlowchart, sendStatus



moderateOrHighUv A step of the program

UV 3 to 7: flashes and beeps slowly until A is pressed, then shows how much sunscreen to put on and starts the 2 hour timer. If sunscreen is already on, it just shows a tick and the time left. If nobody presses A within 10 minutes, the device assumes it is not being worn and goes to sleep.

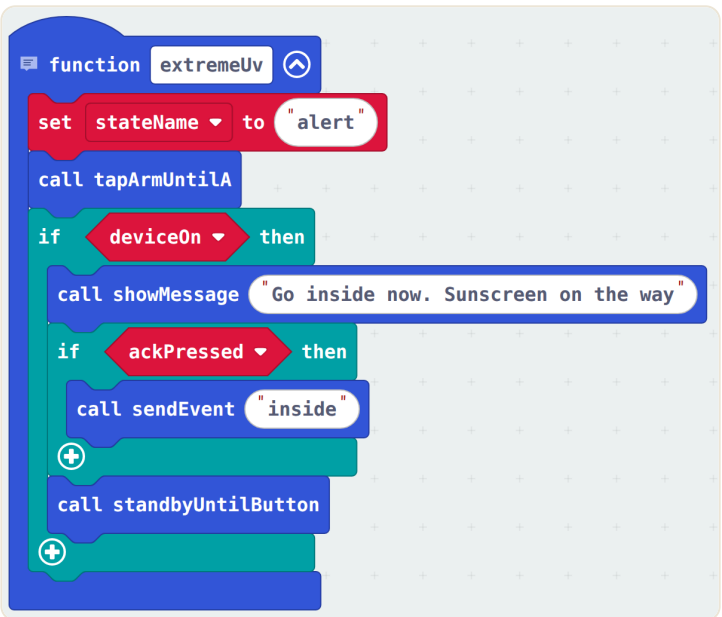
Used by: runFlowchart · **Uses:** sunscreenApplied, alreadyProtected, flashAndBeepUntilA, standbyUntilButton, showMessage



veryHighUv A step of the program

UV 8 to 10: flashes and beeps quickly until A is pressed, then shows the sunscreen advice plus hat and shade, and starts the 2 hour timer. It alerts again if the wearer so far only had the moderate advice.

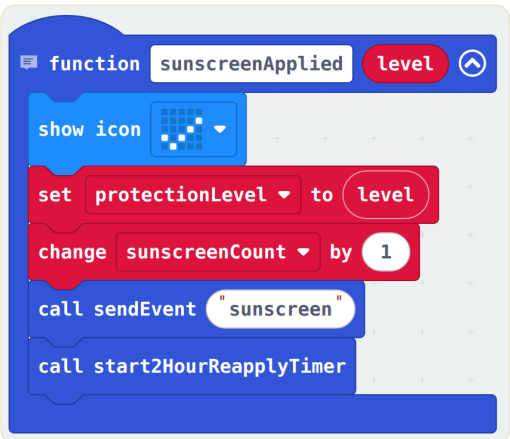
Used by: runFlowchart · **Uses:** sunscreenApplied, alreadyProtected, flashAndBeepUntilA, standbyUntilButton, showMessage



extremeUv A step of the program

UV 11 or more: taps the arm until A is pressed (after 2 minutes it beeps instead), shows Go inside now, then sleeps until a button is pressed.

Used by: runFlowchart · **Uses:** tapArmUntilA, standbyUntilButton, showMessage, sendEvent



sunscreenApplied A step of the program · needs: level

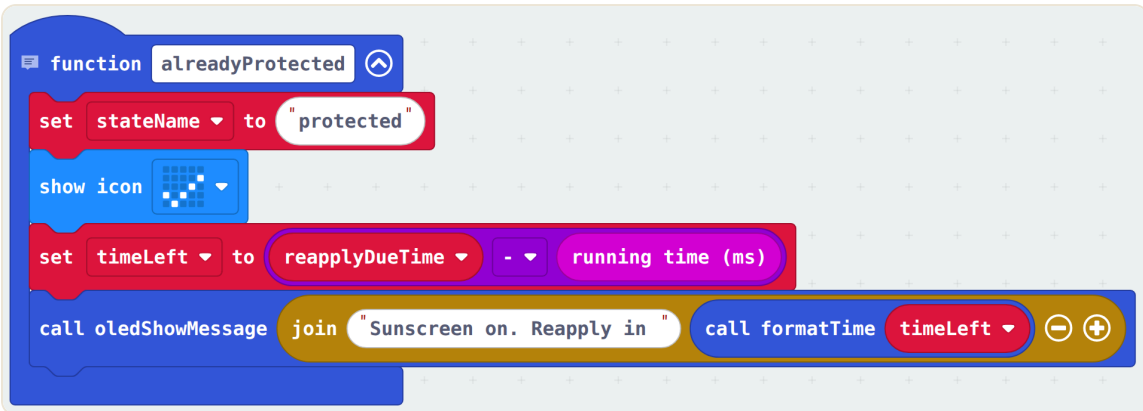
A was pressed after a sunscreen alert: remembers which advice was given, counts one more application, tells the app, and starts the timer.

Used by: moderateOrHighUv, veryHighUv · **Uses:** start2HourReapplyTimer, sendEvent

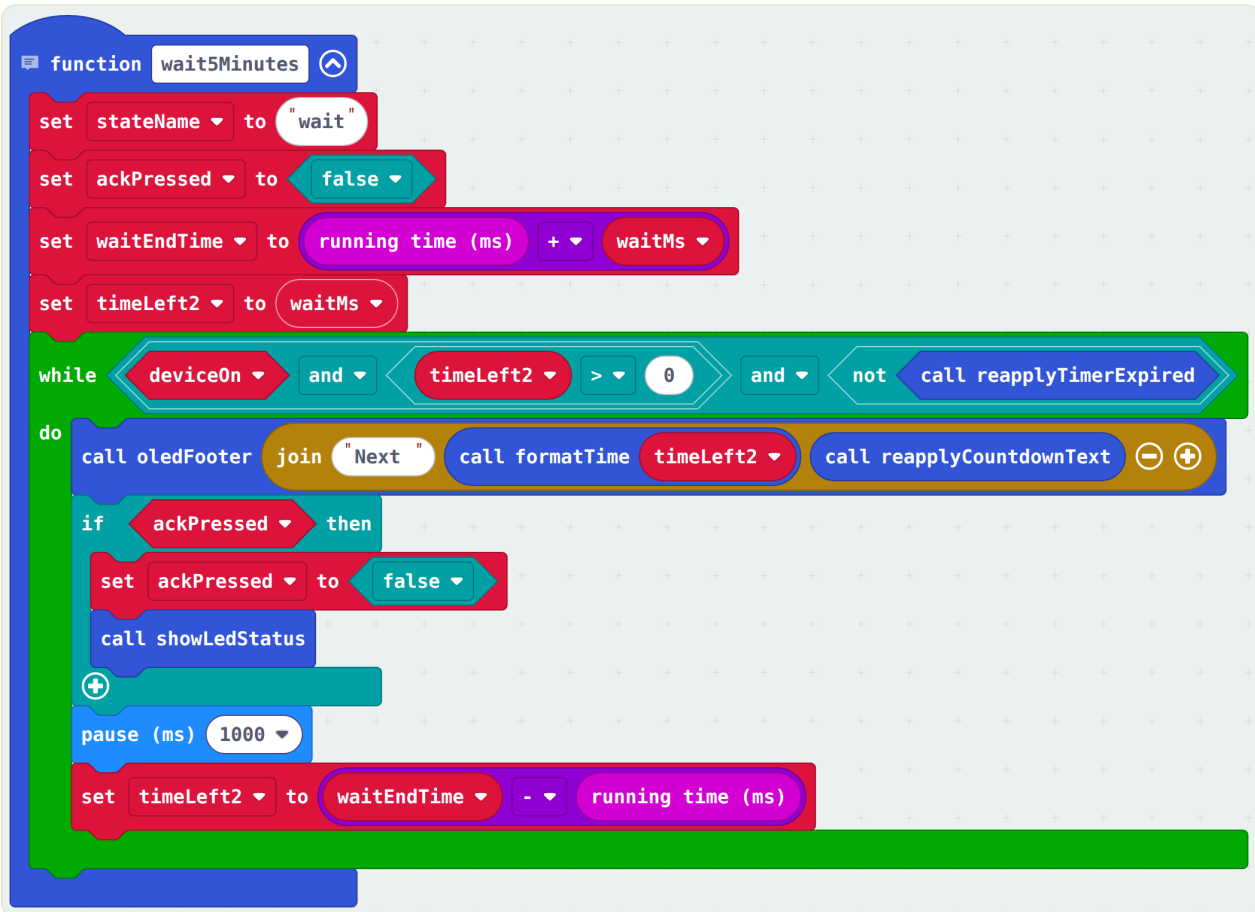
2 The flowchart (continued)

20 blocks

The plan the device follows all day, taken straight from the Sunburn Flowchart. Every box and every question on the flowchart is one block here, with the same name, in the same order: read the UV sensor, check the reading makes sense, ask the phone for the online UV index, decide how strong the sun is, then react. The forever block keeps the plan running round and round.



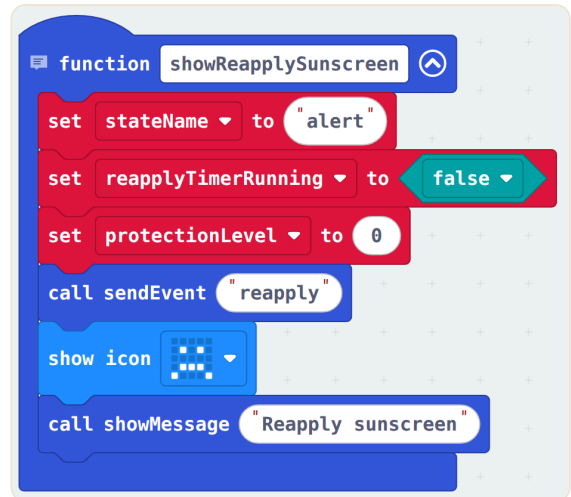
alreadyProtected A step of the program
Sunscreen is already on, so there is no alert: just a tick and the time until it needs reapplying.
Used by: moderateOrHighUv, veryHighUv · **Uses:** oledShowMessage, formatTime



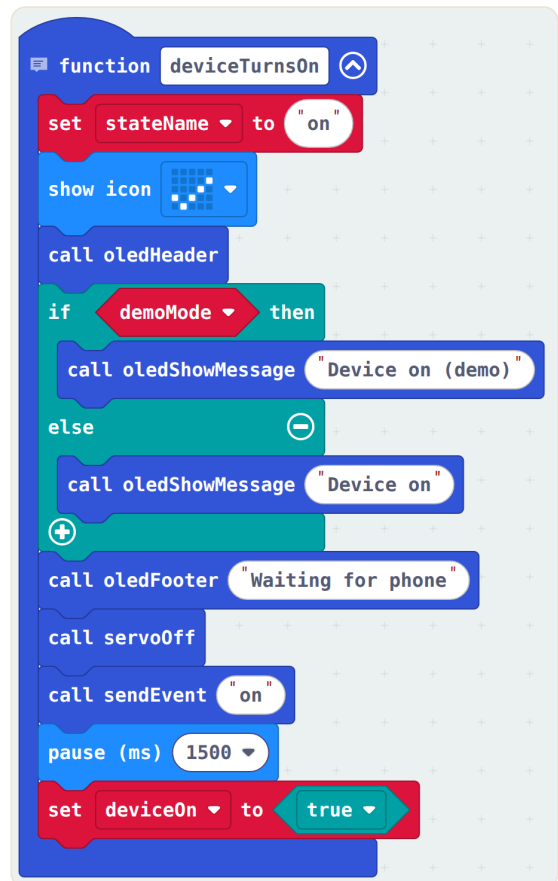
wait5Minutes A step of the program
Waits 5 minutes before the next check. The wait ends early if the device is turned off or the sunscreen timer runs out. Pressing A during the wait shows the UV number and the countdown.
Used by: runFlowchart · **Uses:** reapplyTimerExpired, showLedStatus, oledFooter, formatTime, reapplyCountdownText



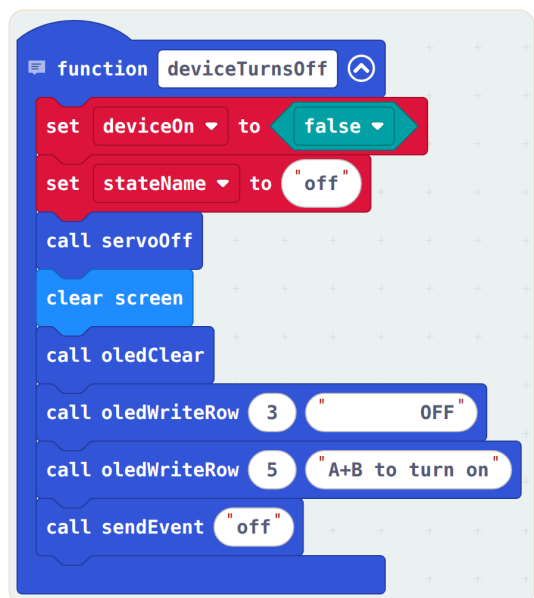
reapplyTimerExpired A yes-or-no question the program asks
Asks: have the 2 hours since the last sunscreen passed?
Used by: runFlowchart, wait5Minutes



showReapplySunscreen A step of the program
Shows Reapply sunscreen and clears the timer, so the next trip round the flowchart alerts again if the sun is still strong.
Used by: runFlowchart · **Uses:** showMessage, sendEvent



deviceTurnsOn A step of the program
Switching on: shows a tick, says Device on on the screen, tells the app, and marks the device as on.
Used by: on button A+B pressed, handleCommand, on start · **Uses:** oledHeader, oledShowMessage, oledFooter, servoOff, sendEvent

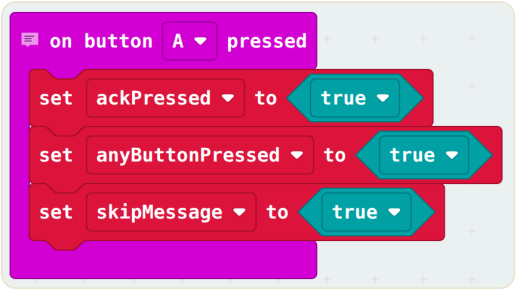


deviceTurnsOff A step of the program
Switching off: everything stops and the screen shows OFF. Bluetooth stays on so the app can switch it back on.
Used by: on button A+B pressed, handleCommand · **Uses:** servoOff, sendEvent, oledClear, oledWriteRow

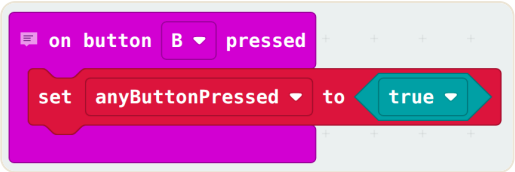
3 Buttons

3 blocks

The micro:bit has two buttons. A means done: sunscreen is on, or I am going inside. B wakes the device up. Pressing both together turns it off, or on again.



on button A pressed Runs each time button A is pressed
Pressing A means done: sunscreen is on, or I am going inside. It also wakes the device, skips a long message, and during a wait shows the UV number and the countdown.



on button B pressed Runs each time button B is pressed
Pressing B wakes the device from sleep. Holding B while switching on gives the fast demo timings.



on button A+B pressed Runs when A and B are pressed together
Pressing A and B together turns the device off, or on again.
Uses: deviceTurnsOn, deviceTurnsOff

The ways the device gets your attention: flashing and beeping, tapping your arm with the small motor, and going to sleep when nobody answers. Each one keeps going until you press A or its time limit runs out. No answer for a long time means the device is probably not being worn, so it sleeps until a button is pressed.

```
function flashAndBeepUntilA fast
  set ackPressed to false
  set beatMs to 500
  set tone to 523
  if fast then
    set beatMs to 150
    set tone to 659
  +
  set giveUpTime to running time (ms) + alertGiveUpMs
  while call stillWaitingForA giveUpTime
  do
    call ledsAllOn
    call beep tone beatMs
    clear screen
    pause (ms) beatMs
```

flashAndBeepUntilA A step of the program · needs: fast
Flashes all 25 lights and beeps until A is pressed or the app says done. The fast setting is the quicker, higher beep for very high UV. It gives up after 10 minutes so a device left on a table does not beep all day.
Used by: moderateOrHighUv, veryHighUv, tapArmUntilA · **Uses:** stillWaitingForA, ledsAllOn, beep

```
function stillWaitingForA untilTime
  return not ackPressed and deviceOn and running time (ms) < untilTime
```

stillWaitingForA A yes-or-no question the program asks · needs: untilTime
Asks: should the alert carry on? Yes while A has not been pressed, the device is still on, and the time limit has not been reached.
Used by: flashAndBeepUntilA, tapArmUntilA

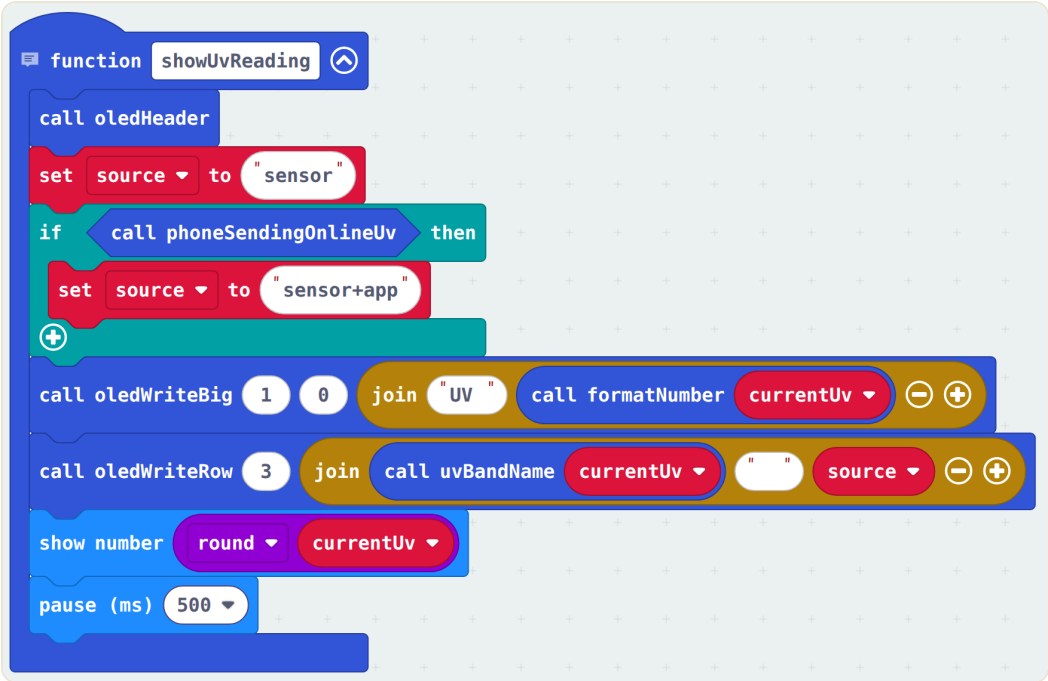
```
function tapArmUntilA
  set ackPressed to false
  show icon
  set stopTappingTime to running time (ms) + tapMaxMs
  while call stillWaitingForA stopTappingTime
  do
    call servoTapOnce
  call servoOff
  if not ackPressed and deviceOn then
    call flashAndBeepUntilA true
  +
```

tapArmUntilA A step of the program
Taps the wearer's arm with the motor until A is pressed. After 2 minutes it beeps instead, so a device that is not being worn does not run the motor flat.
Used by: extremeUv · **Uses:** flashAndBeepUntilA, stillWaitingForA, servoOff, servoTapOnce

```
function standbyUntilButton
  set wentToStandby to true
  set stateName to "standby"
  call sendEvent "standby"
  set anyButtonPressed to false
  show icon
  call oledShowMessage "Standby. Press a button to check again"
  call oledFooter ""
  while deviceOn and not anyButtonPressed
  do
    pause (ms) 200
```

standbyUntilButton A step of the program
Sleep: waits for any button, or done from the app. Pressing A and B together still turns the device off. Afterwards it goes straight back to reading the sensor.
Used by: moderateOrHighUv, veryHighUv, extremeUv · **Uses:** oledShowMessage, oledFooter, sendEvent

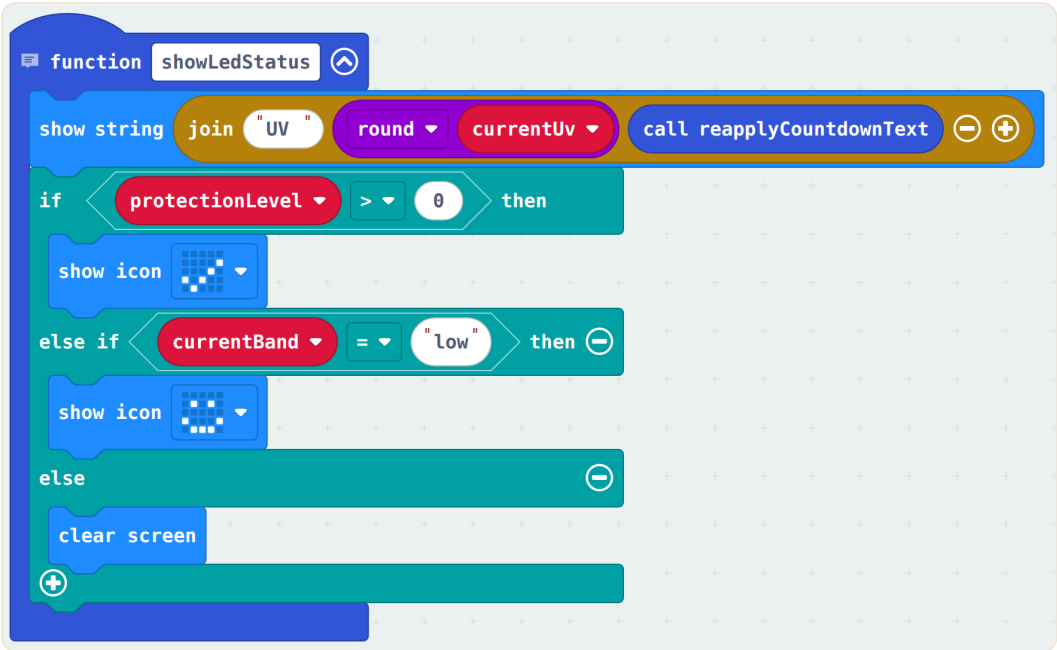
What you see. The 5 by 5 grid of lights shows the UV number, a happy face or a warning, and messages one word at a time. If a small screen is fitted, it shows the number large, the level, the message and the countdowns.



showUvReading A step of the program
Shows the UV number: rounded on the grid of lights, and exact, with the level and where it came from, on the screen.
Used by: runFlowchart · **Uses:** phoneSendingOnlineUv, uvBandName, oledHeader, formatNumber, oledWriteRow, oledWriteBig



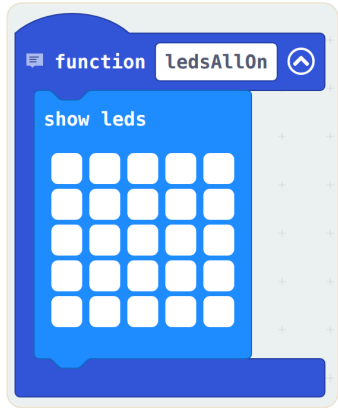
uvBandName Works out a piece of text · needs: uv
The words for the screen: LOW, MODERATE, HIGH, VERY HIGH or EXTREME.
Used by: showUvReading



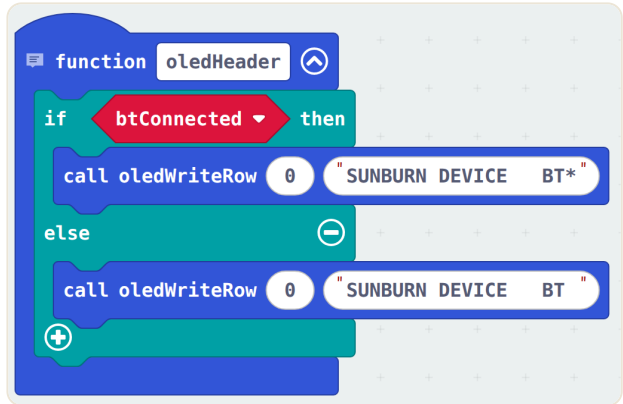
showLedStatus A step of the program
When A is pressed during a wait: the UV number and the time until reapply scroll across the lights, then the usual icon comes back.
Used by: wait5Minutes · **Uses:** reapplyCountdownText



oledFooter A step of the program · needs: text
The bottom line of the screen: the countdowns.
Used by: showCheckSensor, wait5Minutes, deviceTurnsOn, standbyUntilButton · **Uses:** oledWriteRow



ledsAllOn A step of the program
Turns all 25 lights on: the flash of an alert.
Used by: flashAndBeepUntilA

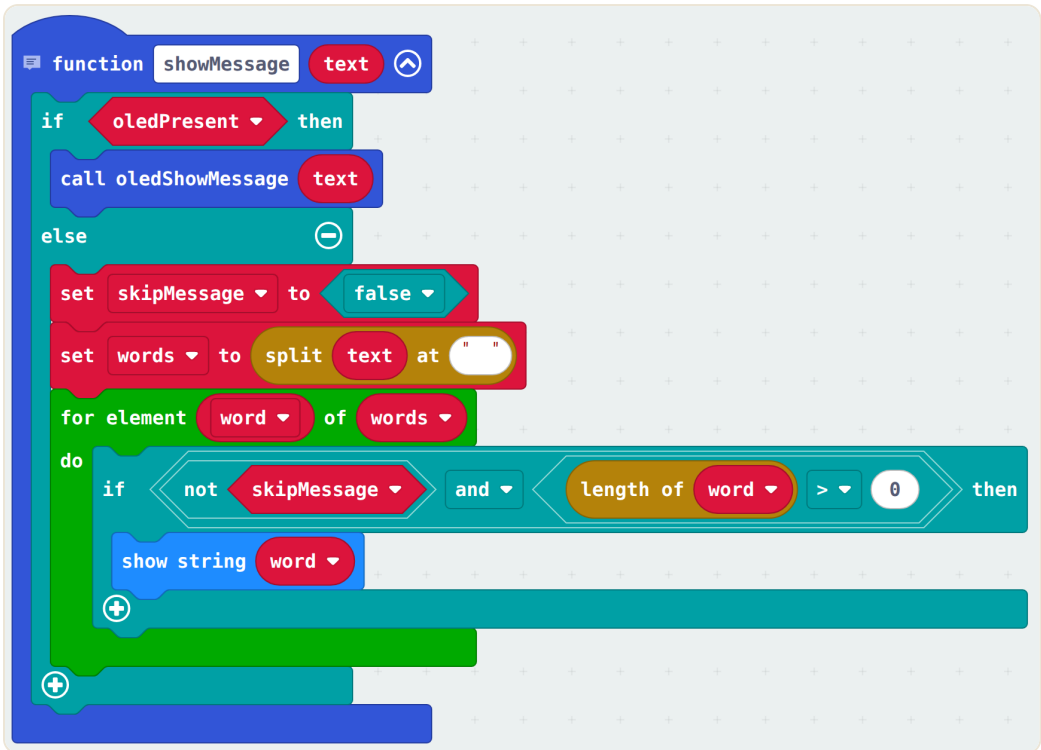


oledHeader A step of the program
The top line of the screen: the title, and a star while a phone is connected.
Used by: deviceTurnsOn, showUvReading, on bluetooth connected, on bluetooth disconnected · **Uses:** oledWriteRow

5 Screen and lights (continued)

8 blocks

What you see. The 5 by 5 grid of lights shows the UV number, a happy face or a warning, and messages one word at a time. If a small screen is fitted, it shows the number large, the level, the message and the countdowns.



showMessage A step of the program · needs: text

Shows a message: on the screen if there is one, otherwise scrolled across the lights one word at a time. Pressing A skips the rest.

Used by: showCheckSensor, moderateOrHighUv, veryHighUv, extremeUv, showReapplySunscreen · **Uses:** oledShowMessage



oledShowMessage A step of the program · needs: text

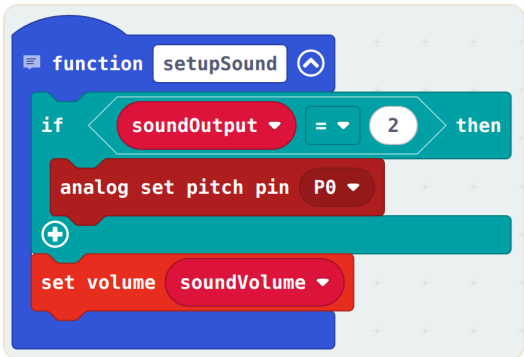
The middle of the screen: a message, split over three lines of 21 letters.

Used by: showHappyFace, alreadyProtected, deviceTurnsOn, standbyUntilButton, showMessage · **Uses:** oledWriteRow

6 Sound and the tapping arm

5 blocks

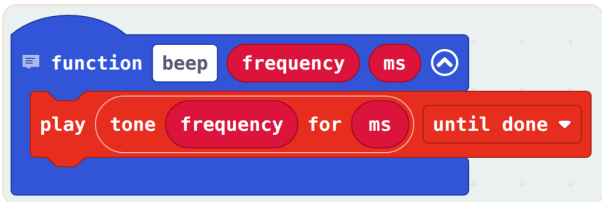
The beeps, and the small motor (a servo) whose arm taps your wrist when the sun is extreme.



setupSound A step of the program

Gets the sound ready: the micro:bit's built-in speaker, or a buzzer on pin P0, at the volume from the settings.

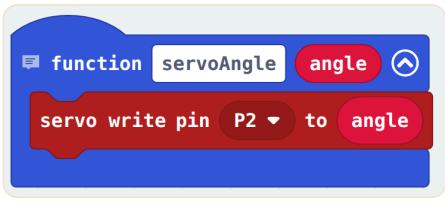
Used by: on start



beep A step of the program · needs: frequency, ms

One beep. It needs how high the beep should sound (the frequency) and how long it lasts (in milliseconds).

Used by: showCheckSensor, flashAndBeepUntilA

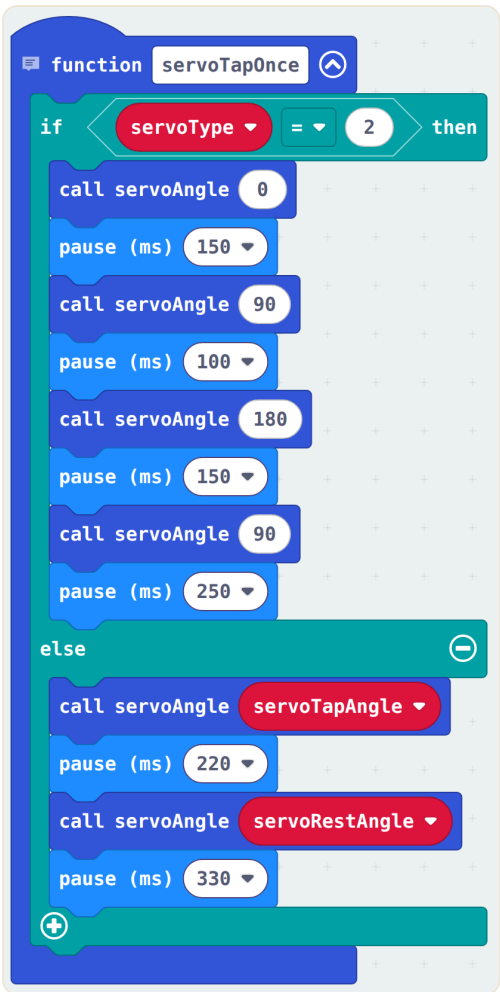


servoAngle A step of the program ·

needs: angle

Turns the motor's arm to an angle. The motor's signal wire is on pin P2.

Used by: servoOff, servoTapOnce



servoTapOnce A step of the program

One tap of the arm, about half a second long.

Used by: tapArmUntilA · **Uses:** servoAngle



servoOff A step of the program

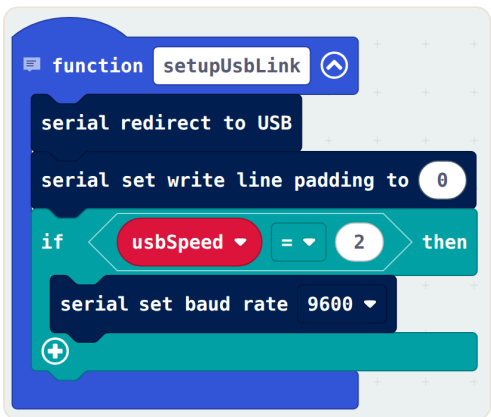
Moves the arm back to rest, then stops sending it signals so it is silent and uses no power.

Used by: deviceTurnsOn, deviceTurnsOff, tapArmUntilA · **Uses:** servoAngle

7 Talking to the phone

12 blocks

How the wearable and the phone app talk over Bluetooth. The phone sends the online UV index for where you are; the wearable sends back what it is doing every two seconds and reports each event, such as sunscreen applied. The same messages also go down the USB cable, so everything can be tested on a laptop. Every message ends with a checksum, so a message damaged on the way is thrown away instead of being misread.



setupUsbLink A step of the program

Gets the USB link to a laptop ready: the same messages as Bluetooth go down the cable, without padding spaces, and at the slow speed if the settings ask for it.

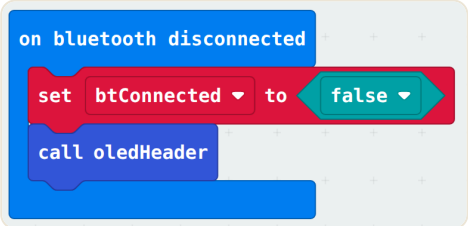
Used by: on start



advertiseOverBluetooth A step of the program

Makes the wearable visible over Bluetooth so a phone can connect.

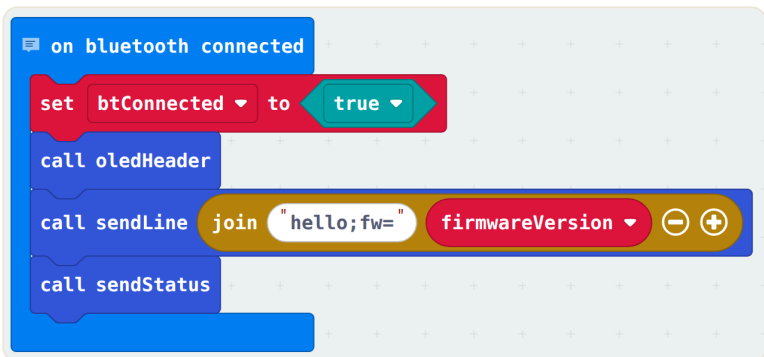
Used by: on start



on bluetooth disconnected Runs when the phone goes away

The phone has gone: remember that, and update the screen.

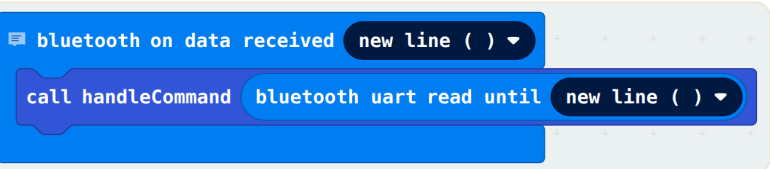
Uses: oledHeader



on bluetooth connected Runs when a phone connects

A phone has connected: say hello and send it the current status straight away.

Uses: oledHeader, sendLine, sendStatus



on bluetooth data received Runs when a line of text arrives from the phone

A line of text arrived from the phone: pass it to the block that reads commands.

Uses: handleCommand

7

Talking to the phone (continued)

12 blocks

How the wearable and the phone app talk over Bluetooth. The phone sends the online UV index for where you are; the wearable sends back what it is doing every two seconds and reports each event, such as sunscreen applied. The same messages also go down the USB cable, so everything can be tested on a laptop. Every message ends with a checksum, so a message damaged on the way is thrown away instead of being misread.

```
serial on data received
  new line ( )
  call handleCommand
  serial read until
    new line ( )
```

on serial data received Runs when a line of text arrives over the USB cable
A line of text arrived over the USB cable: pass it to the block that reads commands.
Uses: handleCommand

```
every 2000 ms
  call sendStatus
```

every 2000 ms Runs every 2 seconds in the background
Every 2 seconds, send the app a status line.
Uses: sendStatus

```
function sendEvent
  name
  call sendLine
    join "ev=" name
```

sendEvent A step of the program · needs: name
Tells the app something happened: sunscreen applied, a reminder, went inside, sleep, a sensor problem, on or off.
Used by: extremeUv, sunscreenApplied, showReapplySunscreen, deviceTurnsOn, deviceTurnsOff, standbyUntilButton · **Uses:** sendLine

```
function sendLine
  text
  set line2 to
    join text ";ck="
    call lineChecksum text
  serial write line line2
  if btConnected then
    bluetooth uart write line line2
```

sendLine A step of the program · needs: text
Sends one message, with its checksum on the end, down the USB cable and to the phone when one is connected.
Used by: showCheckSensor, on bluetooth connected, sendEvent, sendStatus, handleCommand, calibrateToKnownUv · **Uses:** lineChecksum

```
function calibrateToKnownUv
  knownUv
  call readUvSensor
  if knownUv > 0 and sensorVolts > uvZeroVolts then
    set uvVoltsPerIndex to
      sensorVolts - uvZeroVolts / knownUv
    call sendLine "ok=cal"
  else
    call sendLine "err=cal"
```

calibrateToKnownUv A step of the program · needs: knownUv
Corrects the sensor's scale. With the sensor in the sun, the app sends today's real UV index and the wearable adjusts its maths so it reads the same.
Used by: handleCommand · **Uses:** readUvSensor, sendLine

7 Talking to the phone (continued)

12 blocks

How the wearable and the phone app talk over Bluetooth. The phone sends the online UV index for where you are; the wearable sends back what it is doing every two seconds and reports each event, such as sunscreen applied. The same messages also go down the USB cable, so everything can be tested on a laptop. Every message ends with a checksum, so a message damaged on the way is thrown away instead of being misread.

```
function sendStatus
  set reapplySeconds to -1
  if reapplyTimerRunning then
    set msLeft to reapplyDueTime - running time (ms)
    set reapplySeconds to max of 0 and floor msLeft / 1000
  +
  set onlineText to "-1"
  if call phoneSendingOnlineUv then
    set onlineText to call formatNumber onlineUv
  +
  set status to join "st=" stateName - +
  set status to join status ";uv=" call formatNumber currentUv - +
  set status to join status ";sen=" call formatNumber max of 0 and sensorUv - +
  set status to join status ";onl=" onlineText - +
  set status to join status ";band=" call uvBand currentUv - +
  set status to join status ";spf=" reapplySeconds - +
  set status to join status ";spf=" sunscreenCount - +
  set status to join status ";fw=" firmwareVersion - +
  if demoMode then
    set status to join status ";demo=1" - +
  else
    set status to join status ";demo=0" - +
  +
  set status to join status ";pv=" call formatNumber sensorVolts - +
  call sendLine status
```

sendStatus A step of the program

Builds the status line the app shows: what the device is doing, the UV values, the level, the seconds until sunscreen is due, how many applications so far, the program version, whether demo timings are on, and the voltage on the sensor pin.

Used by: on bluetooth connected, every 2000 ms, handleCommand · **Uses:** phoneSendingOnlineUv, uvBand, sendLine, formatNumber

```
function handleCommand command
  set parts to split call cleanText command at "="
  set key to parts get value at 0
  if length of array parts > 1 then
    set value to parts get value at 1
  +
  call debugLog join "cmd" key "value" - +
  if key = "uv" then
    set newUv to parse to number value
    if newUv ≥ 0 and newUv ≤ 20 then
      set onlineUv to newUv
      set onlineUvTime to running time (ms)
      call sendLine "ok=uv"
    else
      call sendLine "err=uv"
  +
  else if key = "ack" then
    set ackPressed to true
    set anyButtonPressed to true
    call sendLine "ok=ack"
  else if key = "zero" then
    call readUvSensor
    set uvZeroVolts to sensorVolts
    call sendLine join "ok=zero;v=" call formatNumber uvZeroVolts - +
  else if key = "cal" then
    call calibrateToKnownUv parse to number value
  else if key = "demo" then
    set demoMode to value = 1
    call applyTimings
    call sendLine "ok=demo"
  else if key = "power" then
    if value = 0 and deviceOn then
      call deviceTurnsOff
    else if value = 1 and not deviceOn then
      call deviceTurnsOn
  +
  else if key = "debug" then
    set debugMode to value = 1
    call sendLine "ok=debug"
  else if key = "vol" then
    set soundVolume to parse to number value
    set volume soundVolume
    call sendLine "ok=vol"
  else if key = "ping" then
    set oledText to 0
    if oledPresent then
      set oledText to 1
  +
  call sendLine join "pong;fw=" firmwareVersion ";oled=" oledText - +
  else if key = "read" then
    call sendStatus
  else if length of key > 0 then
    call sendLine "err=unknown"
  +
```

handleCommand A step of the program · needs: command

Reads a command from the app or a laptop and acts on it: a new online UV value, done, calibrate the sensor, demo timings on or off, switch off or on, extra detail on or off, the beep volume, ping, or send the status now.

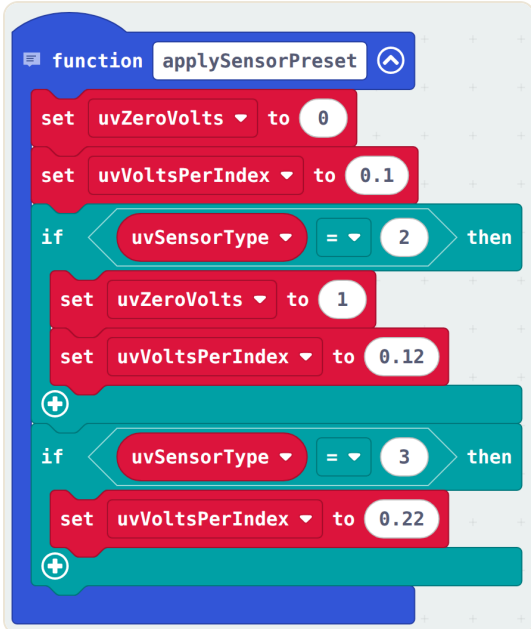
Used by: on bluetooth data received, on serial data received ·

Uses: readUvSensor, deviceTurnsOn, deviceTurnsOff, sendLine, sendStatus, calibrateToKnownUv, applyTimings, formatNumber, cleanText, debugLog

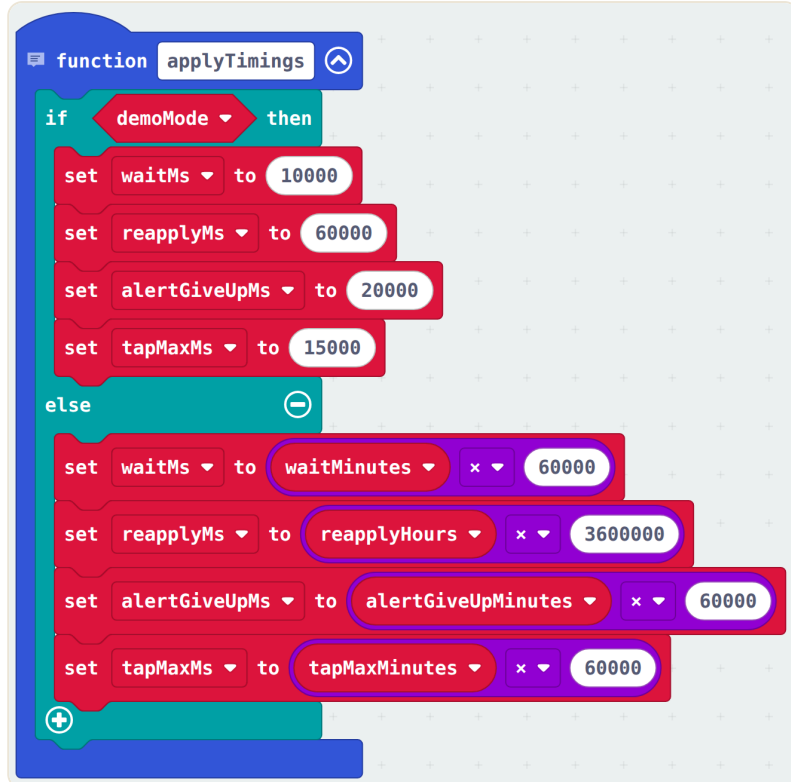
8 Helpers

11 blocks

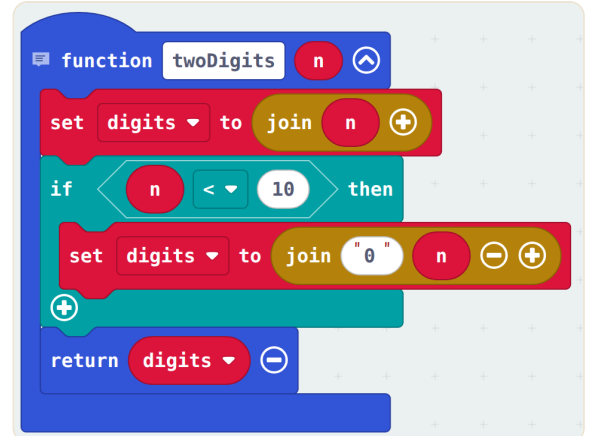
Small jobs the other blocks share: the maths for each type of sensor, turning minutes into the milliseconds the program counts in, sorting numbers, tidying up numbers, times and text, and the checksum that goes on the end of every message.



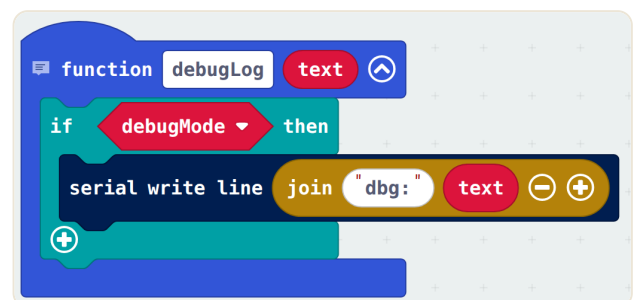
applySensorPreset A step of the program
The maths for each type of UV sensor: which voltage means zero, and how many volts make one point of UV index.
Used by: on start



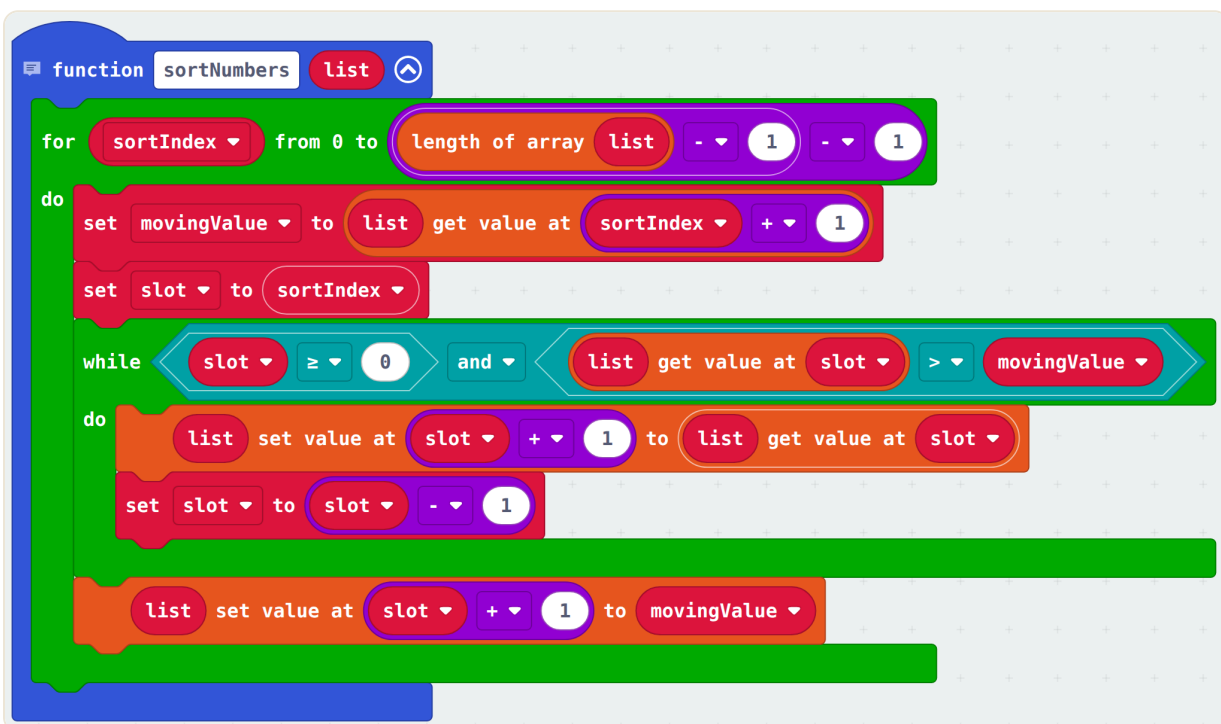
applyTimings A step of the program
Turns the minutes and hours from the settings into the milliseconds the program counts in. Demo timings are much shorter, so the whole flowchart can be shown in a few minutes.
Used by: handleCommand, on start



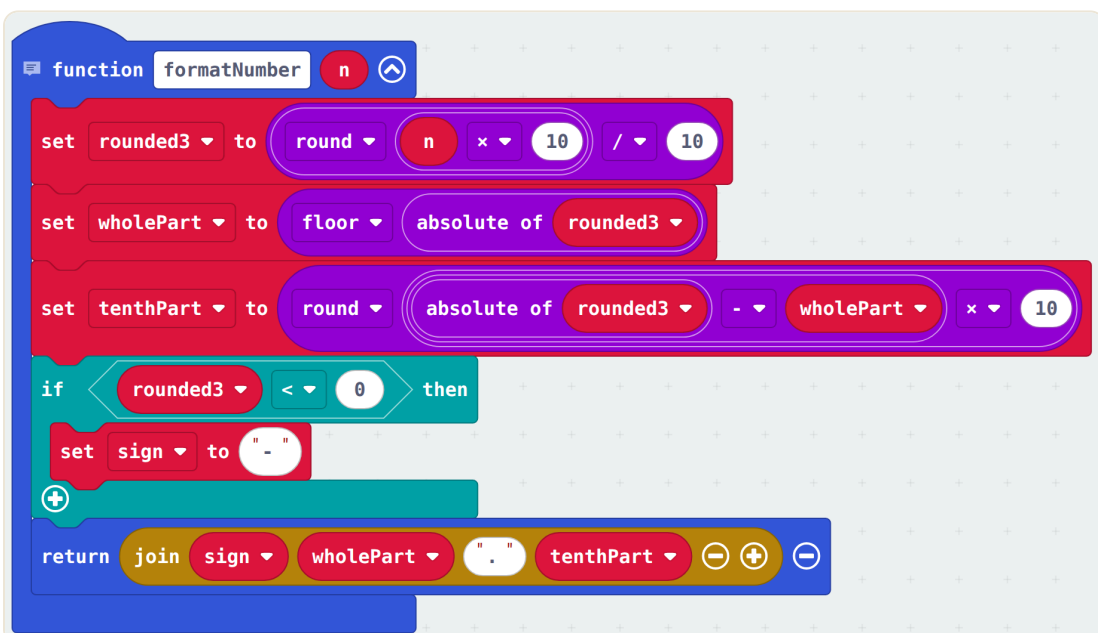
twoDigits Works out a piece of text · needs: n
Adds a leading zero: 7 becomes 07.
Used by: formatTime



debugLog A step of the program · needs: text
Extra detail lines for a laptop, sent only after the command debug=1.
Used by: readUvSensor, handleCommand

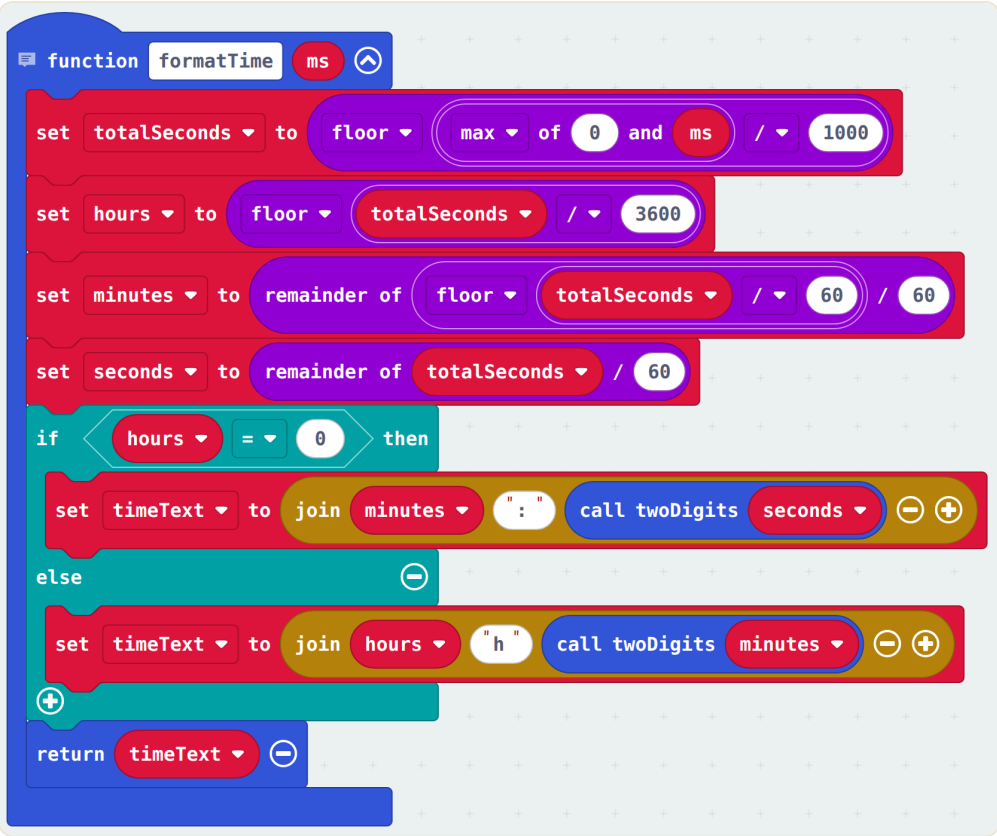


sortNumbers A step of the program · needs: list
Puts a list of numbers in order from smallest to largest. It is used to find the middle sensor reading.
Used by: readUvSensor



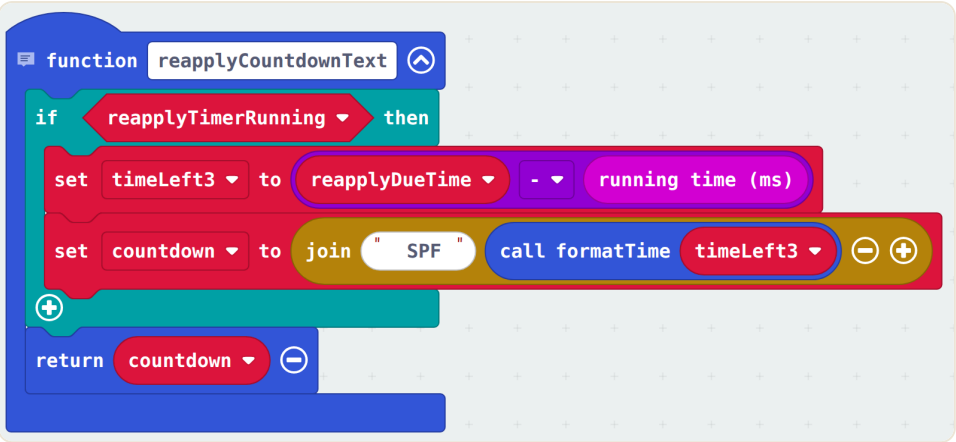
formatNumber Works out a piece of text · needs: n
Writes a number with one decimal place: 7.25 becomes 7.3.
Used by: showCheckSensor, showUvReading, sendStatus, handleCommand

Small jobs the other blocks share: the maths for each type of sensor, turning minutes into the milliseconds the program counts in, sorting numbers, tidying up numbers, times and text, and the checksum that goes on the end of every message.



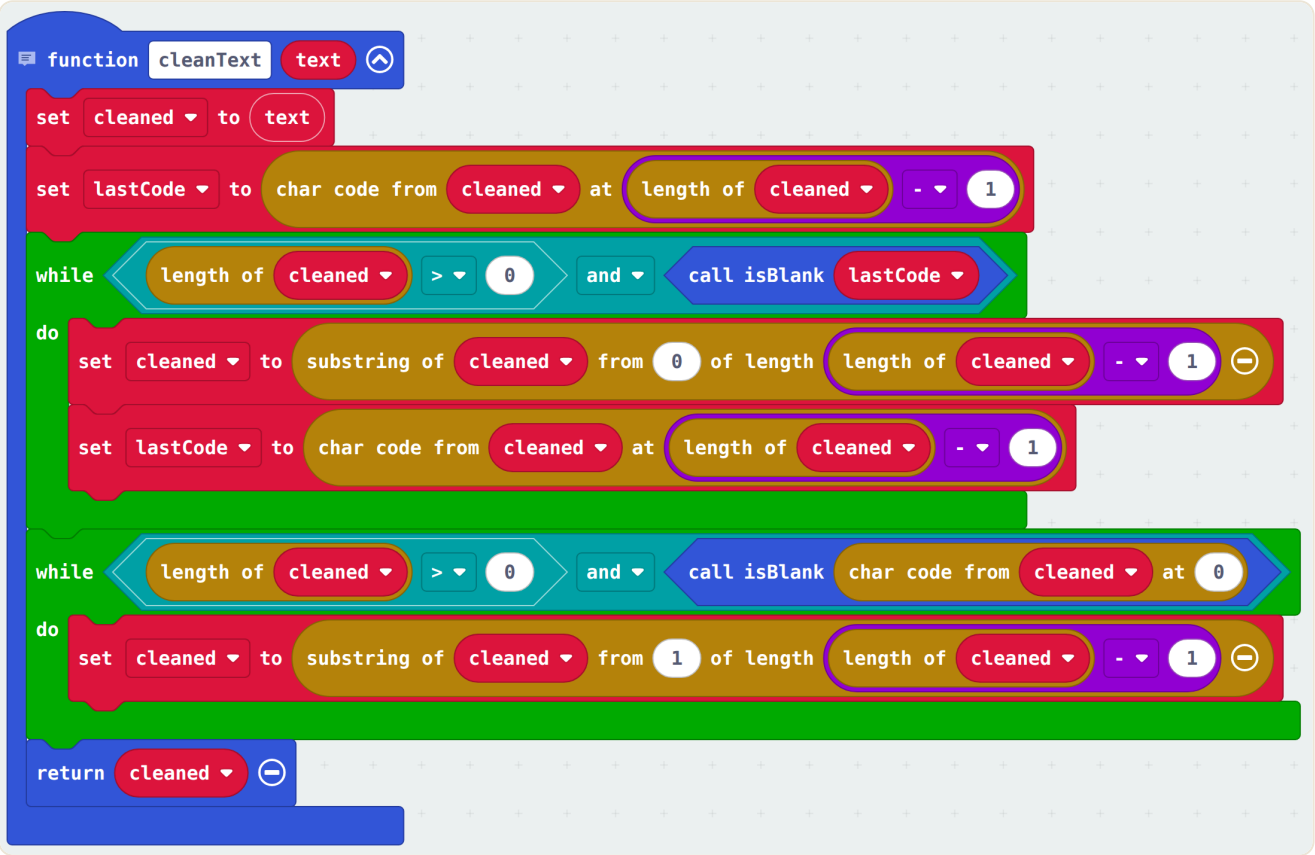
```
function formatTime ms
  set totalSeconds to floor max of 0 and ms / 1000
  set hours to floor totalSeconds / 3600
  set minutes to remainder of floor totalSeconds / 60 / 60
  set seconds to remainder of totalSeconds / 60
  if hours = 0 then
    set timeText to join minutes ":" call twoDigits seconds
  else
    set timeText to join hours "h" call twoDigits minutes
  return timeText
```

formatTime Works out a piece of text · needs: ms
Writes a time the way a clock would: 1:30, or 2h00.
Used by: alreadyProtected, wait5Minutes, reapplyCountdownText · **Uses:** twoDigits



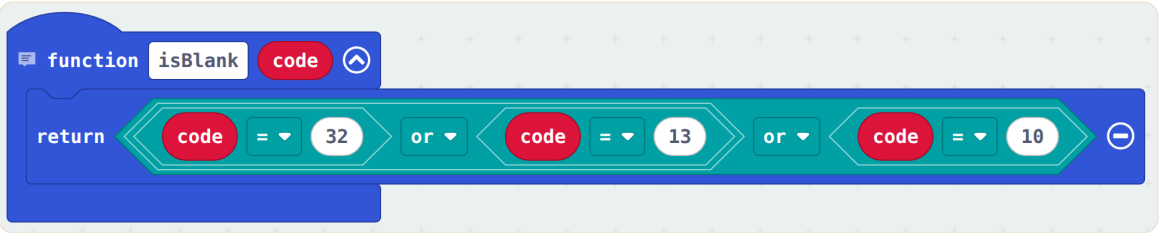
```
function reapplyCountdownText
  if reapplyTimerRunning then
    set timeLeft3 to reapplyDueTime - running time ms
    set countdown to join " SPF " call formatTime timeLeft3
  return countdown
```

reapplyCountdownText Works out a piece of text
The short SPF 1h23 text for the bottom of the screen while the sunscreen timer is running.
Used by: wait5Minutes, showLedStatus · **Uses:** formatTime



```
function cleanText text
  set cleaned to text
  set lastCode to char code from cleaned at length of cleaned - 1
  while length of cleaned > 0 and call isBlank lastCode
    do
      set cleaned to substring of cleaned from 0 of length length of cleaned - 1
      set lastCode to char code from cleaned at length of cleaned - 1
  while length of cleaned > 0 and call isBlank char code from cleaned at 0
    do
      set cleaned to substring of cleaned from 1 of length length of cleaned - 1
  return cleaned
```

cleanText Works out a piece of text · needs: text
Removes stray spaces and line endings from a command before it is read.
Used by: handleCommand · **Uses:** isBlank



```
function isBlank code
  return code = 32 or code = 13 or code = 10
```

isBlank A yes-or-no question the program asks · needs: code
Asks: is this character a space or the end of a line?
Used by: cleanText

8

Helpers (continued)

11 blocks

Small jobs the other blocks share: the maths for each type of sensor, turning minutes into the milliseconds the program counts in, sorting numbers, tidying up numbers, times and text, and the checksum that goes on the end of every message.

```
function lineChecksum text
  for index from 0 to length of text - 1
  do
    set sum to remainder of sum + char code from text at index / 256
  return sum
```

lineChecksum Works out a number · needs: text
Adds up the character codes of a message (0 to 255). It goes on the end of every message, so the app can tell when a message was damaged on the way and ignore it.
Used by: sendLine

9

The optional screen

12 blocks

Only used if a small screen is added to the wearable. These blocks draw letters and numbers on it dot by dot. The device works exactly the same without a screen.

```
function setupOled
  set oledPresent to false
  if oledMode > 0 then
    call loadFont
    call buildNibbleTable
    call oledCommand 174
    set screenStatus to i2c read number at address oledAddress of format UInt8LE repeated false
    if oledMode = 2 or screenStatus = 64 then
      call oledInitCommands
      set oledPresent to true
      call oledClear
```

setupOled A step of the program
Looks for a screen on pins 19 and 20 and gets it ready. If there is no screen, everything else works the same.
Used by: on start · **Uses:** oledInitCommands, loadFont, buildNibbleTable, oledCommand, oledClear

```
function fontByte code column
  set safeCode to code
  if safeCode < 32 or safeCode > 126 then
    set safeCode to 63
  return fontBytes get value at safeCode - 32 * 5 + column
```

fontByte Works out a number · needs: code, column
One column of dots, out of five, of one character.
Used by: oledWriteText, oledWriteBig

```
function oledCommand command
  i2c write number
  at address oledAddress
  with value command
  of format UInt16BE
  repeated false
```

oledCommand A step of the program
· needs: command
Sends the screen one instruction.
Used by: setupOled, oledInitCommands, oledSetPosition

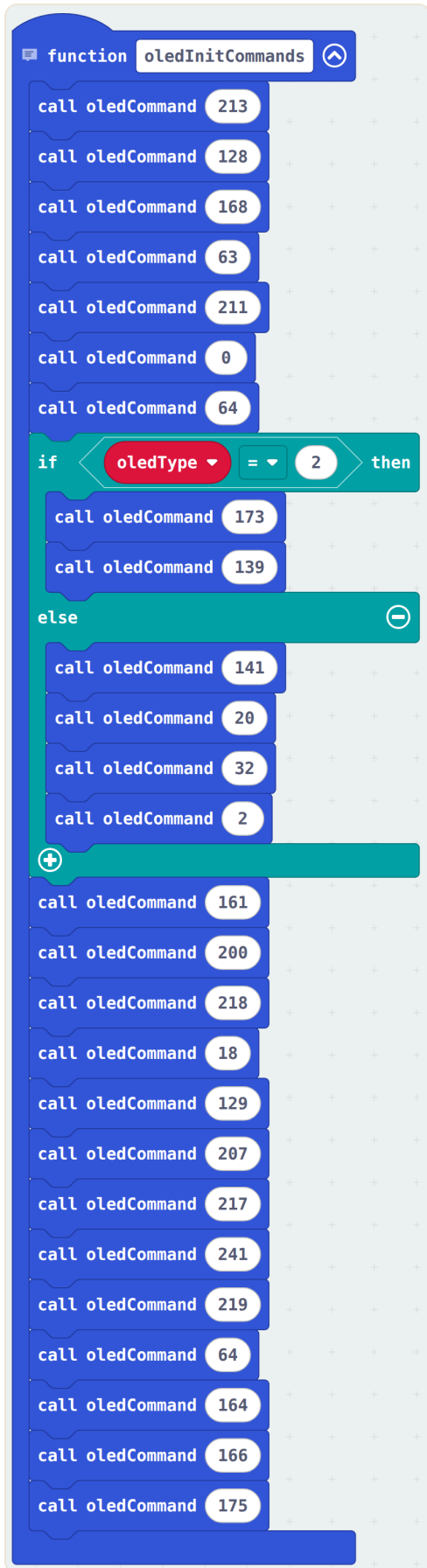
```
function oledData byte1 byte2 byte3
  set packet to byte1 * 65536
  change packet by byte2 * 256
  change packet by byte3
  i2c write number
  at address oledAddress
  with value 1073741824 + packet
  of format UInt32BE
  repeated false
```

oledData A step of the program · needs: byte1, byte2, byte3
Sends the screen three columns of dots.
Used by: oledClear, oledWriteText, oledWriteBig

9 The optional screen (continued)

12 blocks

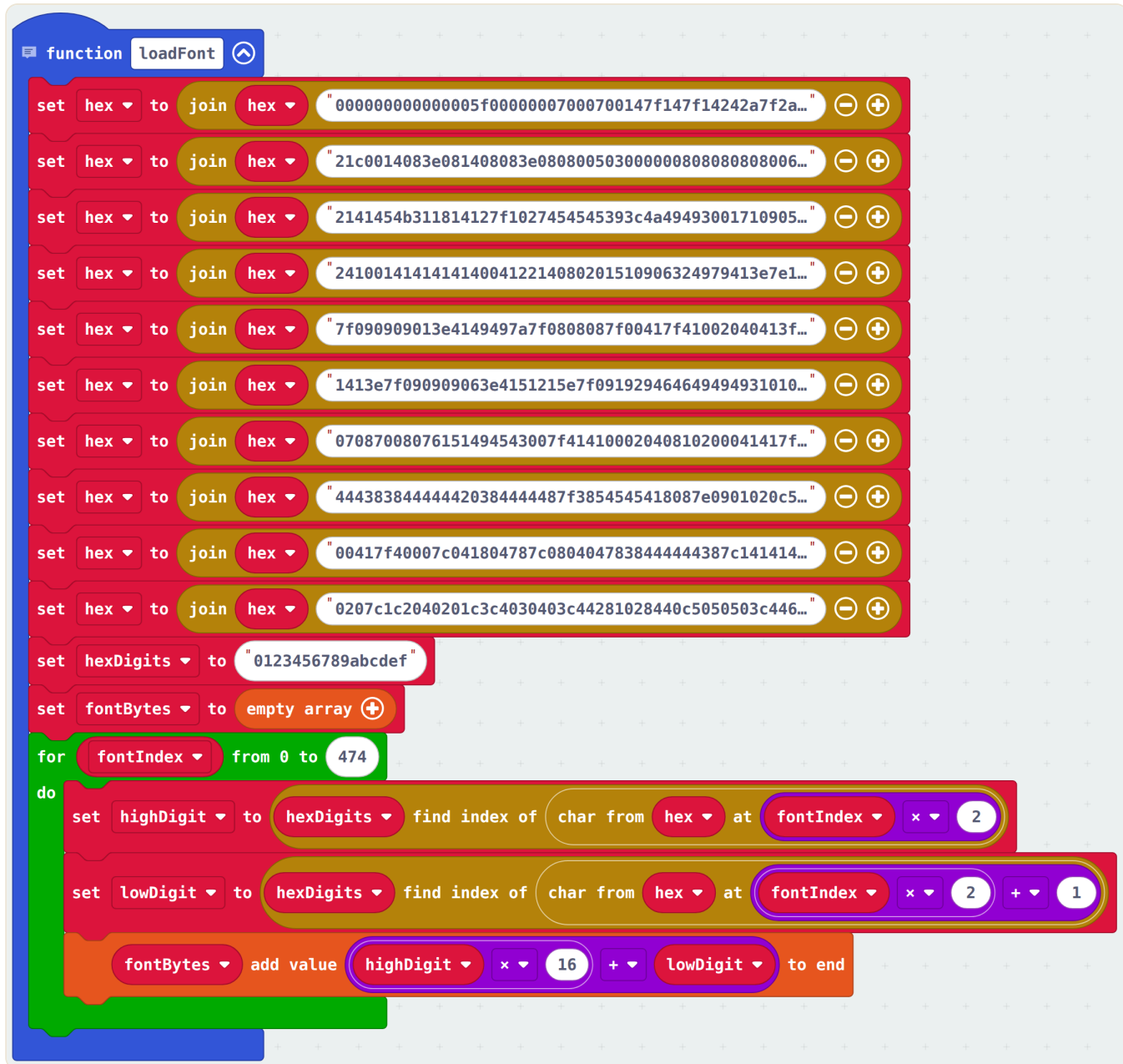
Only used if a small screen is added to the wearable. These blocks draw letters and numbers on it dot by dot. The device works exactly the same without a screen.



oledInitCommands A step of the program

The standard list of instructions that wakes up a 128 by 64 dot screen.

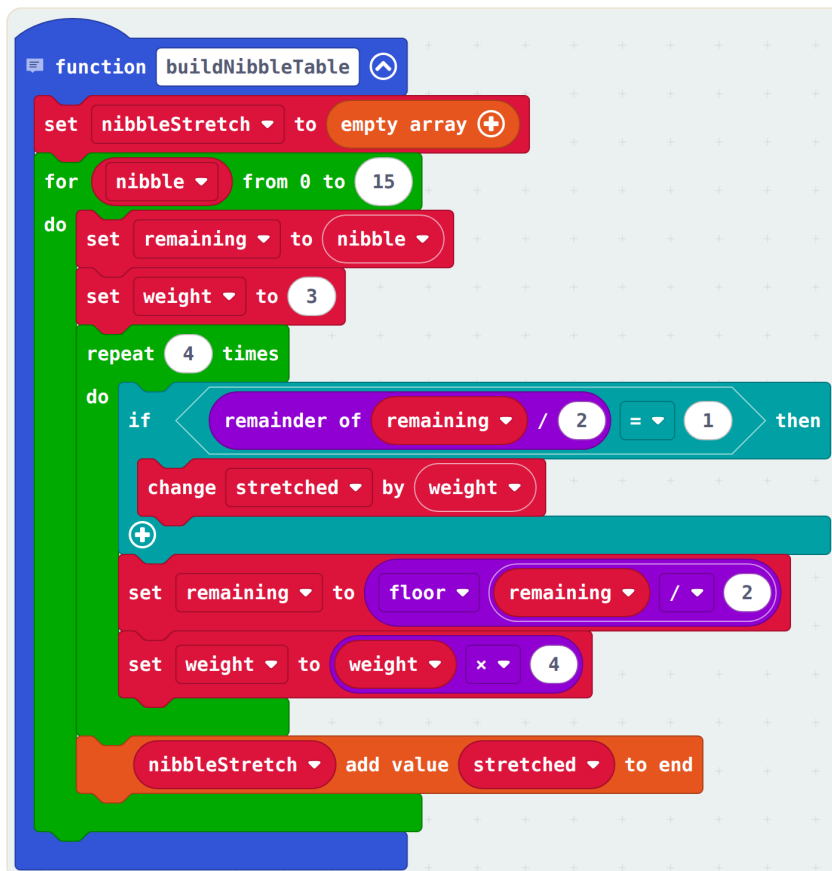
Used by: setupOled · **Uses:** oledCommand



loadFont A step of the program

The shape of every letter, number and symbol, 5 dots wide by 7 tall, stored as text and unpacked when the device starts.

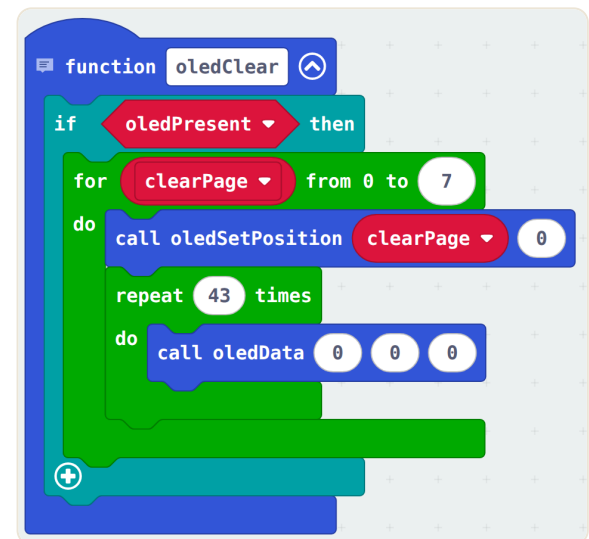
Used by: setupOled



buildNibbleTable A step of the program

A lookup table for double-size letters: every dot is doubled sideways.

Used by: setupOled



oledClear A step of the program

Blanks the whole screen.

Used by: deviceTurnsOff, setupOled · **Uses:** oledData, oledSetPosition

9

The optional screen (continued)

12 blocks

Only used if a small screen is added to the wearable. These blocks draw letters and numbers on it dot by dot. The device works exactly the same without a screen.

```
function oledSetPosition page col
  set pixelColumn to col
  if oledType = 2 then
    change pixelColumn by 2
  call oledCommand 176 + page
  call oledCommand remainder of pixelColumn / 16
  call oledCommand 16 + floor pixelColumn / 16
```

oledSetPosition A step of the program · needs: page, col
Moves to a place on the screen: a row from 0 to 7 and a column from 0 to 127.
Used by: oledClear, oledWriteText, oledWriteBig · **Uses:** oledCommand

```
function oledWriteText row col text
  if oledPresent then
    set smallText to text
    if length of smallText > 21 - col then
      set smallText to substring of smallText from 0 of length 21 - col
    call oledSetPosition row col x 6
    for charIndex from 0 to length of smallText - 1
      do
        set charCode to char code from smallText at charIndex
        call oledData call fontByte charCode 0 call fontByte charCode 1 call fontByte charCode 2
        call oledData call fontByte charCode 3 call fontByte charCode 4 0
```

oledWriteText A step of the program · needs: row, col, text
Writes small text: up to 21 characters on each of 8 rows.
Used by: oledWriteRow · **Uses:** fontByte, oledData, oledSetPosition

```
function oledWriteRow row text
  set rowText to text
  while length of rowText < 21
    do
      set rowText to join rowText " "
  call oledWriteText row 0 rowText
```

oledWriteRow A step of the program · needs: row, text
Writes a whole row, filling the rest with spaces so old text is wiped.
Used by: deviceTurnsOff, showUvReading, oledHeader, oledShowMessage, oledFooter · **Uses:** oledWriteText

9 The optional screen (continued)

12 blocks

Only used if a small screen is added to the wearable. These blocks draw letters and numbers on it dot by dot. The device works exactly the same without a screen.

```
function oledWriteBig row col text
  if oledPresent then
    set bigText to text
    if length of bigText > 10 then
      set bigText to substring of bigText from 0 of length 10
    for half from 0 to 1
      do
        call oledSetPosition row + half col x 6
        for bigCharIndex from 0 to length of bigText - 1
          do
            set bigCharCode to char code from bigText at bigCharIndex
            for columnIndex from 0 to 5
              do
                if columnIndex < 5 then
                  set glyphColumn to call fontByte bigCharCode columnIndex
                if half = 0 then
                  stretchedColumns add value nibbleStretch get value at remainder of glyphColumn / 16 to end
                else
                  stretchedColumns add value nibbleStretch get value at floor glyphColumn / 16 to end
            call oledData stretchedColumns get value at 0 stretchedColumns get value at 0 stretchedColumns get value at 1
            call oledData stretchedColumns get value at 1 stretchedColumns get value at 2 stretchedColumns get value at 2
            call oledData stretchedColumns get value at 3 stretchedColumns get value at 3 stretchedColumns get value at 4
            call oledData stretchedColumns get value at 4 stretchedColumns get value at 5 stretchedColumns get value at 5
```

oledWriteBig A step of the program · needs: row, col, text
Writes double-size text, two rows tall and up to 10 characters. It is used for the big UV number.
Used by: showUvReading · **Uses:** fontByte, oledData, oledSetPosition